© 2011 Lijuan Luo

NEW STRATEGIES FOR ELECTRONIC DESIGN AUTOMATION PROBLEMS

BY

LIJUAN LUO

DISSERTATION

Submitted in partial fulfillment of the requirements
for the degree of Doctor of Philosophy in Electrical and Computer Engineering
in the Graduate College of the
University of Illinois at Urbana-Champaign, 2011

Urbana, Illinois

Doctoral Committee:

Professor Martin D.F. Wong, Chair
Professor Wen-mei W. Hwu
Professor Janak H. Patel
Assistant Professor Deming Chen

ABSTRACT

As the semiconductor industry marches towards 22 nm technology and beyond, circuit design has become unprecedentedly complicated. This presents many new challenges for EDA (electronic design automation), such as lack of effective tools for analog circuit or high-volume and high-frequency printed circuit board (PCB) design, the contradiction between complex EDA compute workloads and time-to-market pressure, manufacturing variability and power management, to name but a few. In this dissertation, we will propose several new strategies to handle the challenges in the EDA field.

Wire routing is an important step in the design of PCBs. Although there are many industrial tools to handle IC routing problems, very few tools can handle the routing on high-density and high-frequency boards effectively. Nowadays, most of the PCB routing is still done by tedious and time-consuming manual work. We provide new strategies to solve an important problem in PCB routing, the escape routing problem. Our first strategy is to use Boolean satisfiability to optimally solve the escape routing problem on one PCB component. Our second strategy is to use a novel boundary routing methodology to finish escape routing from two connected PCB components simultaneously. This router can achieve much better routability than industrial tools with less CPU time.

Another challenge seen in the EDA field is the increasing CPU time to handle larger and larger designs. On the other hand, many fundamental algorithms and data structures used in the EDA tools have shown great parallelism, such as the well-known BFS (breadth-first search) algorithm and the R-tree structure. Therefore, we propose strategies to use the cost-effective GPU platform to parallelize and accelerate BFS and R-tree query. These strategies are potentially applicable to many EDA problems.

TABLE OF CONTENTS

CHAPTER 1 INTRODUCTION 1

CHAPTER 2 SAT FOR ESCAPE ROUTING 5
2.1 Introduction 5
2.2 SAT Formulation 6
2.3 Handling Infeasible Problems 18
2.4 Handling Large Problems 19
2.5 Experimental Results 24
2.6 Conclusion 27

CHAPTER 3 BOUNDARY ROUTING FOR SIMULTANEOUS ESCAPE 32
3.1 Introduction 32
3.2 Boundary Routing 34
3.3 Dynamic Net Ordering 40
3.4 Application to PCB Routing 43
3.5 Solve Large Problems by Clustering 46
3.6 Experimental Results 50
3.7 Conclusion 54

CHAPTER 4 GPU IMPLEMENTATION OF BFS 55
4.1 Introduction 55
4.2 BFS and Previous Works 56
4.3 GPU Architecture 58
4.4 Our GPU Solution 60
4.5 Experimental Results 66
4.6 Conclusion 70

CHAPTER 5 GPU IMPLEMENTATION OF R-TREE 71
5.1 Introduction 71
5.2 Background 72
5.3 Parallel R-tree Query 76
5.4 Parallel R-tree Construction 82
5.5 Experimental Results 84
5.6 Application to Via Dropping 85

5.7 Conclusion 89

CHAPTER 6 CONCLUSION 90

REFERENCES 92

CHAPTER 1

INTRODUCTION

With the rapid development of VLSI technology, we are facing more and more challenging designs. Take the printed circuit board (PCB) design for example; it used to be perfectly solved by manual work or simple heuristic algorithms. Now with tens of components, thousands of pins and more than ten signal net layers on a high-end board, simple heuristic algorithms can hardly provide satisfying designs. And manual work becomes more and more time-consuming and tedious. The first part of this thesis focuses on new algorithms to solve the PCB design problems, particularly the component escape routing problems. On the other hand, the developing technology has brought us new computing power. The graphics processing unit (GPU) has evolved in recent years to become a more general processor, allowing users to flexibly program certain aspects of the GPU for data-parallel and computing intensive applications. In the second part of the thesis, we will present new techniques to accelerate two fundamental EDA algorithms on the GPU.

A PCB contains several components such as multi-chip modules (MCMs), memory and I/O modules. These components are either mounted or plugged into the board such that each component pin is accessible from every layer of the board. The PCB routing problem is to determine the wiring connection between signal pins of different components. The routing is always done in a planar fashion on each layer since introducing vias can affect signal integrity and lower manufacturing yield. In addition, several design constraints need to be enforced during routing, such as minimum/maximum length constraints, adjacency/separation constraints and differential pair constraints. Due to the complexity of the PCB routing problem, it is typically decomposed into two subproblems (Fig. 1.1), the *escape routing* which routes nets from pins to their component boundaries and the *area routing* which routes nets between component boundaries. The major task of the escape routing is to "escape" the pins from the components and provide a net ordering along the

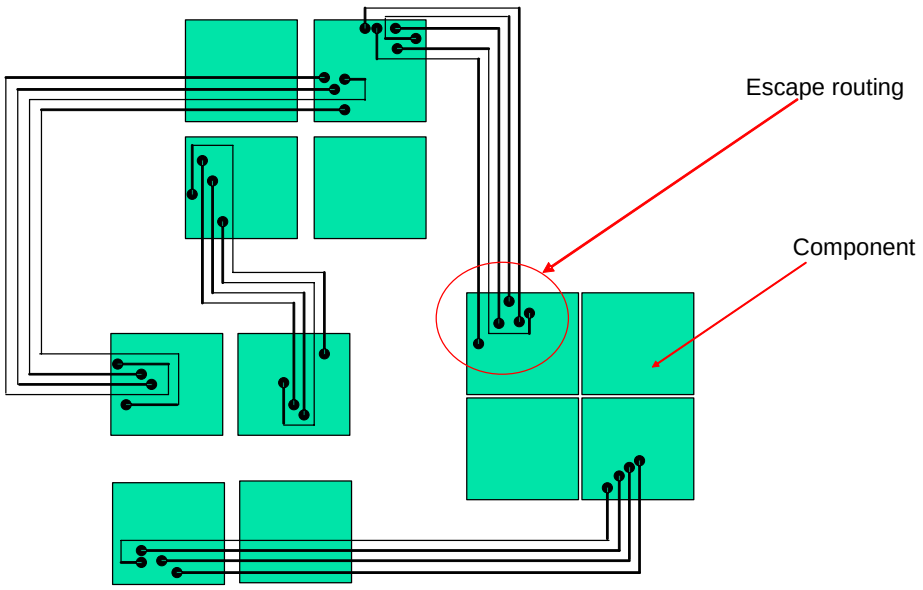


Figure 1.1: The PCB escape routing problem.

boundaries of the two components so that a planar topology can be guaranteed for area routing. The focus of the area routing, on the other hand, is to carefully detour the wires to meet the length constraints while maintaining the planar topology inherited from the escape routing. Our research work has been focusing on the escape routing problems.

Moore's law stated that the number of transistors on an integrated circuit would double each year (later revised to doubling every 18 months). Several measures of digital technology are improving at exponential rates related to Moore's law, including transistor size, density, cost, etc. As the transistor size becomes smaller and the circuit function becomes more complicated, we have seen thousands of pins on very small packages. For example, an MCM of IBM z9 enterprise class server [1] consists of 16 chips and 2970 signal I/O pins within a 95-mm×95-mm area. Therefore, the escape routing which routes a great number of nets out of the tiny component area becomes extremely challenging and turns into the main bottleneck in terms of overall routability [2]. In Chapter 2, we will present a Boolean satisfiability (SAT) based escape routing tool, which can guarantee to find the escape routing solution for mid-sized problems. We will also present techniques to use the SAT router for larger problems to achieve satisfying routability within reasonable CPU time. In Chapter 3, we will present a new boundary routing methodology, which can handle the escape routing on two components simultaneously. This routing

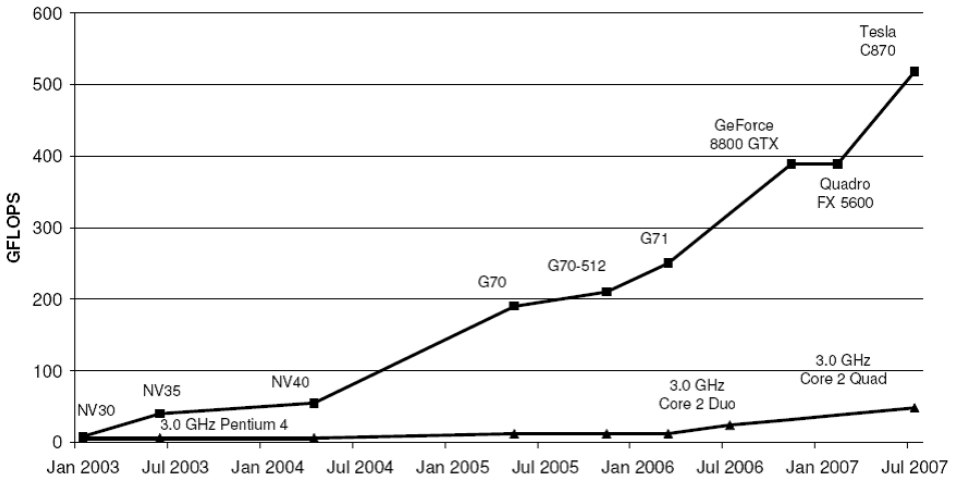


Figure 1.2: Enlarging performance gap between CPUs and GPUs [3]. Each square represents one generation of NVIDIA GPU. Each triangle represents one generation of Intel CPU.

method can achieve a much better routability result than the commercial PCB routing tool within much less CPU time.

The CPU industry has been following Moore's law for over 40 years, doubling the performance of CPUs approximately every two years. However, with the size of transistors gradually approaching the atomic level, the CPU performance cannot continue growing at this speed. On the other hand, the highly-parallel systems such as NVIDIA's CUDA architecture have brought a widening performance gap with traditional x86-based CPU processors in recent years (Fig. 1.2). One of the latest NVIDIA GPUs, GeForce GTX 480, is capable of delivering 1.35 TFOPS of single precision performance and 168 GFLOPS of double precision while executing up to 480 simultaneous threads in one low-cost package. The GPUs are also at an advantage in terms of memory bandwidth. GTX 480 is capable of about 177.4 gigabytes per second into the main DRAM, which is about 7x the bandwidth of the latest Intel Core I7 processor. Moreover, the GPUs are more cost-efficient and power-efficient than the CPUs. Therefore, GPUs have gained significant popularity as powerful tools for high-performance computing in applications such as physics simulation, computer vision, options pricing, sorting and search.

Various phases of design automation for today's complex hardware are computationally intensive. In order to meet the time-to-market constraints, there is a growing need to adapt EDA tools to exploit affordable parallel ar-

chitectures. A great number of works demonstrated the use of GPUs in accelerating EDA tools. In SPICE, 32 times speedup was achieved in the transistor model evaluation by using the GPU [4]. Gauda is the first company to use GPUs to accelerate computation for optical proximity correction (OPC) [5]. Torunoglu et al. proposed an inverse lithography solution which uses GPUs as well as CPUs as computation hardware [6]. A Monte Carlo-based statistical static timing analysis algorithm was proposed in [7] which reported 260x speedup on a single GPU and significantly higher projected speedups on multi-GPU platforms. Gulati and Khatri presented a GPU-based fault simulation tool with 35x speedup over a commercial fault simulation tool [8]. Chatterjee et al. proposed GPU-based gate-level logic simulators both in oblivious fashion [9] and event-driven fashion [10]. There have been several successful works using GPUs for power grid analysis [11], [12], [13]. Liu and Hu used GPU parallel programming to achieve 56x speedup on the optimization of gate sizes and threshold voltages. The list can go on. Apparently the progress of GPU technology has opened a new avenue for improving the performance of EDA algorithms.

In our research, we are trying to use GPUs to accelerate graph algorithms and the operations on the spatial data structure. Graph algorithms such as breadth-first search (BFS) are widely used in the EDA field. Unlike a lot of the above-mentioned EDA algorithms, graph algorithms are not a natural fit for the GPU architecture. There have been very few successful works on GPU-based graph algorithms. We managed to achieve reasonable speedup on BFS by designing a special queue structure on the GPU and an effective strategy to control the parallelization overhead. This work will be presented in Chapter 4. The R-tree structure is one of the most promising spatial data structures. It is very useful in VLSI physical design for efficient checking of object overlap. In Chapter 5, we will propose methods to quickly construct and query an R-tree on the GPU. Our parallel solutions on BFS and the R-tree structure can be potentially applied to the EDA tools for routing, timing analysis, logic simulation, design rule checking, etc.

CHAPTER 2

SAT FOR ESCAPE ROUTING

2.1 Introduction

Printed circuit board (PCB) routing is the problem of determining wiring connections between components on the circuit board. This problem has become extremely difficult due to the rapid increase in pin count and density. There are many papers about packaging technologies as well as routing technologies to achieve higher pin density and fewer routing layers [14–17]. In our work, we focus on the problem of routing one layer assuming a reasonable layer assignment has been done. For today's high-end complex PCBs, the routing problem can only be solved by a substantial amount of tedious and time-consuming manual effort. Thus, more research on the design automation of PCB routing is greatly needed. In this chapter, we consider the ordered escape routing problem which is a key problem in board-level routing.

The input to the ordered escape routing problem is a pin array component and a required ordering of escaped pins on the component boundary. The required pin ordering is a way to ensure routing between components can be done in a planar fashion. Without loss of generality, we may assume that the escape pin ordering is 1, 2, 3, $\cdots$, n where n is the number of pins, by suitably renaming the pins if needed. In the escape problem, we are asked to find disjoint paths to route all pins within the pin array to the component boundary while satisfying the escape pin ordering. Fig. 2.1(a) shows an example of the ordered escape problem where all pins are required to escape at the right side of the component. This is a 1-side escape problem. We can also specify escapes to be on 2 sides, 3 sides, or 4 sides. Fig. 2.1(b) shows a 4-side escape problem. (Although 4-side escape is permissible, only 2 sides are actually used in this solution.)

There is a large body of literature [18], [19], [20], [21] on the "unordered" escape routing problem; i.e., there is no constraint on the ordering of the escape pins. It is well known that the unordered problem can be solved optimally by a network flow approach. However, when it comes to ordered escape, all existing approaches [22], [23] to the problem cannot guarantee to find a routing solution even if one exists. (Actually, [22] and [23] only solve a more restricted problem, where each pin has a fixed escape destination. However, we believe their approach can be extended to allow floating escape destinations, but the escape side for each pin has to be fixed.) The algorithm in [22] can only route nets in monotonic (or detour-free) patterns. Note that the solution in Fig. 2.1(a) is non-monotonic (Net 2's route is a detour) and the problem cannot be solved by monotonic routing. The most recent work [23], although it partially uses integer linear programming, heuristically considers non-monotonic routing only when the routing graph has a cycle. Note that [23] cannot find the routing solution in Fig. 2.1(a), where detour is needed for routing resource conflict, even though no cycle exists in the routing graph. Also, the 4-side escape problem in Fig. 2.1(b) cannot be solved by both [22] and [23], since there is no way to know the escape side of each pin in advance.

In this chapter, we first present an algorithm to exactly solve the escape routing problem based on Boolean satisfiability (SAT). Then we present a strategy to apply the SAT router efficiently for large-scale problems with clustering structure. Furthermore, the SAT router is used as a general rip-up and reroute technique to fix the escape solution obtained by any heuristic router. Experimental results on escape problems from industry show that our methods perform well in terms of both speed and routability.

2.2 SAT Formulation

A SAT problem is to find an assignment of variables to staisfy the equation

$$F(x_1, x_2, ..., x_n) = 1 \tag{2.1}$$

where F is a Boolean function with variables $x_1, x_2, \cdots, x_n$. Most often, F is expressed in conjunctive normal form (CNF). A CNF expression is a logical

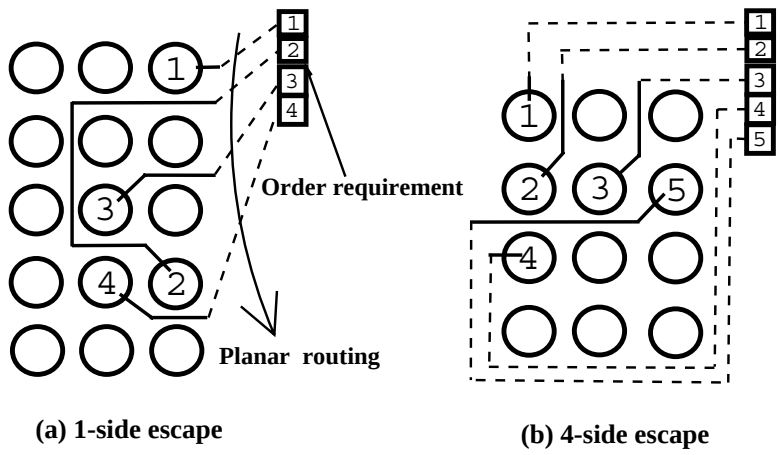


Figure 2.1: Ordered escape routing problem.

and of one or more clauses, where each clause is a logical **or** of one or more literals. A literal is either the positive or the negative form of a variable.

Although the SAT approach has been widely used in CAD applications such as testing and verification [24], [25], [26], [27], [28], its application to physical design problems has been very limited [29], [30]. This is because of the myth that SAT instances in physical layout problems are always too large to be computationally feasible [31]. However, significant progress has been made in SAT solvers such that state-of-the-art SAT solvers can solve CNF formulas of millions of variables and clauses [32]. Moreover, we observe that most of the escape routing problem instances extracted from industrial boards are of moderate sizes, making SAT potentially a viable approach. In this section, we provide a practical approach to optimally solve the escape routing problem using a SAT formulation.

2.2.1 Construct Boolean Functions

For a given pin array, we construct its corresponding routing grid G. (See Fig. 2.2, which is the routing grid for Fig. 2.1(b).) We place additional cells called slots adjacent to the boundary of the routing grid to model the locations of the escape routes exiting the pin array. We define a graph on this routing grid. There are three types of nodes. Each free grid cell is called a grid node, each pin is called a pin node and each slot is called a slot node. An edge (shown as a dashed line in Fig. 2.2) is a connection between two neighboring nodes. We note that due to the physical nature of the pin array,

each pin can only connect "diagonally" to four neighboring grid nodes in the NW, NE, SW, and SE directions, and each pin-slot connection can only be horizontal or vertical.

To formulate the escape routing problem as a SAT problem, we define three types of Boolean variables: grid variable, slot variable and edge variable. For each grid node in G, we introduce a grid variable which is a string of Boolean variables corresponding to the binary representation of the index of the net routing through the node. Note that we can introduce a pin variable for each pin in a similar way except the string of Boolean variables is a constant (equals to the net index of the pin). Similarly, for each slot node, we introduce a slot variable, which is a string of Boolean variables corresponding to the binary representation of the index of the net exiting the pin array at that slot. Finally, an edge variable is a Boolean variable defined for each edge where it is 1 if and only if a net is routed on the edge.

We now show that the escape routing problem can be transformed into a SAT problem of the form

$$F = F_G F_E F_P F_S = 1 \tag{2.2}$$

where G, E, P, and S are the sets of all grid nodes, edges, pin nodes, and slot nodes, respectively. In other words, we are looking for a variable assignment to simultaneously satisfy the equations $F_G = 1$, $F_E = 1$, $F_P = 1$, and $F_S = 1$. Essentially, these equations specify various feasibility conditions on the Boolean variables in a feasible escape routing solution. The details of the constructions of F_G, F_E, F_P, and F_S are given in the following.

1) Construct F_G: For each grid node g, there are at most 8 edges attached to it, corresponding to the connections with 4 neighboring grid nodes and 4 neighboring pin nodes. It is obvious that the number of attached edges cannot be fewer than 2. The variable assignment must satisfy the following condition: Among all the edges attached to g, either no edge variable is equal to 1 or there are exactly two edge variables equal to 1. In other words, either g is not used in routing, or it is used by routing one net from one neighboring edge to another neighboring edge. Let $e_1, \cdots, e_m$ be all the edges attached

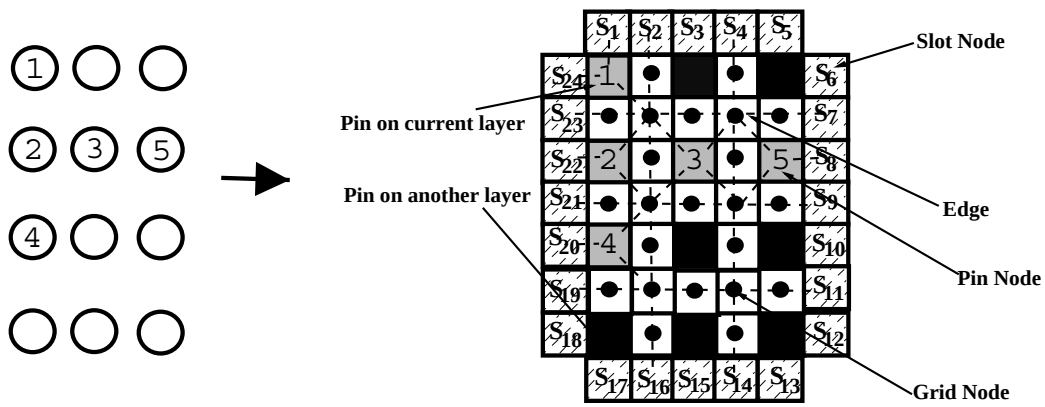


Figure 2.2: Variable definition and slot index.

to node g, $(2 \leq m \leq 8)$. We define a Boolean function f_g for this condition.

$$f_g = (\prod_{i=1}^{m} \overline{e}_i) + (\sum_{\substack{i,j=1 \\ i \neq j}}^{m} (e_i e_j (\prod_{\substack{k=1 \\ k \neq i,j}}^{m} \overline{e}_k))) \tag{2.3}$$

For example, the grid node highlighted in Fig.2.2 has four neighboring edges e_1, e_2, e_3, e_4. For this node, we have

$$\begin{aligned} f_g &= (\overline{e}_1\overline{e}_2\overline{e}_3\overline{e}_3) + (e_1e_2\overline{e}_3\overline{e}_4) + (e_1\overline{e}_2e_3\overline{e}_4) + (e_1\overline{e}_2\overline{e}_3e_4) \\ &\quad +(\overline{e}_1e_2e_3\overline{e}_4) + (\overline{e}_1e_2\overline{e}_3e_4) + (\overline{e}_1\overline{e}_2e_3e_4) \end{aligned} \tag{2.4}$$

Note that in each clause, either no edge variable or exactly two variables are positive.

Letting f_g range over all grid nodes, we get

$$F_G = \prod_{g \in G} f_g \tag{2.5}$$

2) Construct F_E: For each edge e, either the edge variable is equal to 0, or the two nodes connected by e are equal. This is obvious, since the edge cannot be used simultaneously by two nets. Suppose the two node variables connected to e are n_1 and n_2. Define f_e to be the function corresponding to the above requirement. In the following, we also use e to represent the edge variable. Then we have

$$f_e = \overline{e} + (n_1 = n_2) \tag{2.6}$$

Letting f_e range over all edges, we get

$$F_E = \prod_{e \in E} f_e \tag{2.7}$$

3) Construct F_P: For each pin p there should be one and only one neighboring edge equal to 1, corresponding to the edge through which the net routes away from p. Let $e_1, \cdots, e_m$ be all the edges attached to p. We define f_p as follows:

$$f_p = \sum_{i=1}^{m} (e_i (\prod_{\substack{j=1 \\ j \neq i}}^{m} \overline{e}_j)) \tag{2.8}$$

For example, consider Pin 4 in Fig. 2.2, which has three neighboring edges e_1, e_2, e_3; then we have

$$f_p = (e_1 \overline{e}_2 \overline{e}_3) + (\overline{e}_1 e_2 \overline{e}_3) + (\overline{e}_1 \overline{e}_2 e_3) \tag{2.9}$$

Letting f_p range over all pins, we get

$$F_P = \prod_{p \in P} f_p \tag{2.10}$$

4) Construct F_S: This is for satisfying the given escape ordering. We first index the slots in clockwise order starting from the left topmost slot as shown in Fig. 2.2. (The starting point can also be changed to any other slot.) Use S_i to stand for the ith slot, and s_i for the slot variable. Then an escape ordering is correct if and only if for each pair i, j, where $i < j$ and S_i, S_j are both involved in routing, we have $s_i < s_j$. However, since we cannot predict which slots are used in routing in advance, it is hard to come up with a Boolean function to define this order condition. Thus we propose another order condition: An escape order is correct if and only if for each pair i, j, where $i < j$, we have $s_i \leq s_j$. To prove the correctness of the second condition, we compare its differences from the previous one. First, for the two slots used by routing, we require $s_i \leq s_j$ instead of $s_i < s_j$. The two requirements are actually equivalent, since it is not possible for the two slots used by the same net, and $s_i = s_j$ is never true. Second, empty slots (slots not used in routing) are involved in the second condition. This requirement is equivalent to the first condition. Since the variable of an empty slot does not

have any real meaning, we could give it any value. Suppose we get a value assignment which satisfies the first condition, then we can always make this assignment satisfy the second condition by setting the values of the empty slot variables in the following way. If s_i is an empty slot variable, then let $s_i = s_j$, where $j < i$ and S_j is the closest non-empty slot. If S_j does not exist, set $s_i = 0$. Hence, the second condition is totally equivalent to the first one. And now we can define F_S as follows:

$$F_S = \prod_{i=2}^{|S|} (s_{i-1} \leq s_i) \tag{2.11}$$

2.2.2 Transform into CNF

Almost none of the Boolean functions introduced in Section 2.2.1 are expressed in CNF. In order to transform them into concise CNF expressions, we use several techniques, which will be introduced in the following.

1) Negative expression and De Morgan's law

Instead of constructing the Boolean function by enumerating all the legal cases as before, we enumerate all the illegal ones and write a negative expression. For example, for the f_p in (2.9), we have

$$\begin{aligned}
&(e_1\overline{e}_2\overline{e}_3) + (\overline{e}_1 e_2 \overline{e}_3) + (\overline{e}_1\overline{e}_2 e_3) \\
&\Leftrightarrow \overline{(\overline{e}_1\overline{e}_2\overline{e}_3) + (\overline{e}_1 e_2 e_3) + (e_1\overline{e}_2 e_3) + (e_1 e_2 \overline{e}_3) + (e_1 e_2 e_3)} \\
&\Leftrightarrow (e_1 + e_2 + e_3)(e_1 + \overline{e}_2 + \overline{e}_3)(\overline{e}_1 + e_2 + \overline{e}_3) \\
&\quad (\overline{e}_1 + \overline{e}_2 + e_3)(\overline{e}_1 + \overline{e}_2 + \overline{e}_3)\ (De\ Morgan's\ law)
\end{aligned} \tag{2.12}$$

2) Recursive procedure

Some Boolean functions constructed in Section 2.2.1 are based on strings of Boolean variables, such as $a = b$, $a \leq b$, where a and b are two strings of Boolean variables. For convenience of explanation, we assume both of them have only two bits; i.e., $a = a_1 a_2$, $b = b_1 b_2$. The method for transforming $a = b$ to CNF is quite straightforward:

$$\begin{aligned}
a = b \quad &\Leftrightarrow \quad (a_1 = b_1)(a_2 = b_2) \\
&\Leftrightarrow \quad (a_1 + \overline{b}_1)(\overline{a}_1 + b_1)(a_2 + \overline{b}_2)(\overline{a}_2 + b_2)
\end{aligned} \tag{2.13}$$

However, the transformation for $a \leq b$ is much more complex. It not only needs using a negative expression and De Morgan's law, but also a recursive procedure. First, we have

$$a \leq b \Leftrightarrow \overline{a > b} \tag{2.14}$$

We can enumerate all the possible cases for $a > b$. There are only three cases: (1) $a = 1\text{x}$, $b = 0\text{x}$, (2) $a = 11$, $b =\text{x}0$, (3) $a =\text{x}1$, $b = 00$, where "x" means "don't care". (Note that the three cases are not exclusive with each other.) So we have

$$\begin{aligned} a \leq b \quad &\Leftrightarrow \quad \overline{a > b} \\ &\Leftrightarrow \quad \overline{(a_1\overline{b}_1) + (a_1 a_2 \overline{b}_2) + (\overline{b}_1 a_2 \overline{b}_2)} \\ &\Leftrightarrow \quad (\overline{a}_1 + b_1)(\overline{a}_1 + \overline{a}_2 + b_2)(b_1 + \overline{a}_2 + b_2) \end{aligned} \tag{2.15}$$

For a general case, where $a = a_1 \ldots a_k$, $b = b_1 \ldots b_k$, we come up with a recursive procedure to enumerate the three possible cases for $a > b$: (1) $a_1 = 1$, $b_1 = 0$, (2) $a_1 = 1$, $a_2 \ldots a_k > b_2 \ldots b_k$, (3) $b_1 = 0$, $a_2 \ldots a_k > b_2 \ldots b_k$. Based on this recursive procedure, we can get the Boolean function for $a > b$, and then the Boolean function for $a \leq b$.

Once we have transformed the Boolean formula into CNF, we can call a SAT solver to solve the problem. One solution for the problem in Fig. 2.2 is shown in Fig. 2.3. Here, the number in each slot or free grid cell is the Boolean value of the slot variable or the node variable. For conciseness, only the edges with the edge variables equal to 1 are drawn in the figure; the other edges are omitted.

2.2.3 Improvement for Cyclic Ordering

The above SAT formulation is still deficient in resolving 4-side escape cases. The problem is that it takes the boundary as a path, which begins at the left topmost corner. Then it interprets the ordering in a linear way, which means that net 1 should be the first net you meet when you traverse along the path from the starting point. However, the real situation is that the boundary is a cycle with no starting or end point. Fig. 2.4 shows several potential escape patterns for the 4-side escape problem with 4 nets. (Since we only care about the escape ordering at this stage, we omit the internal

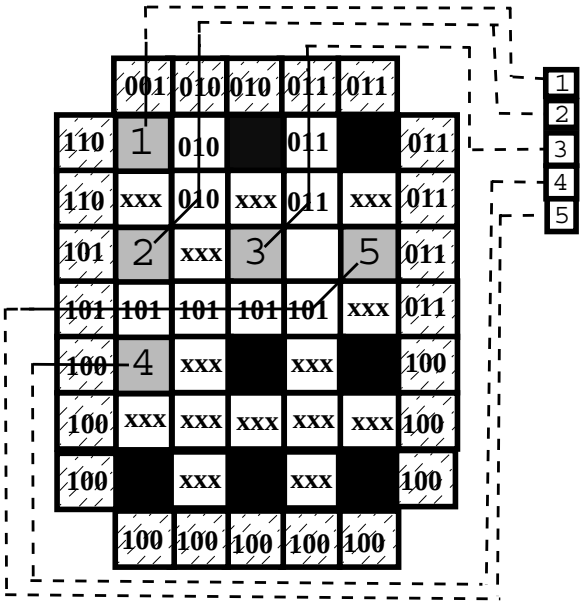


Figure 2.3: SAT solution to the escape problem in Fig. 2.2.

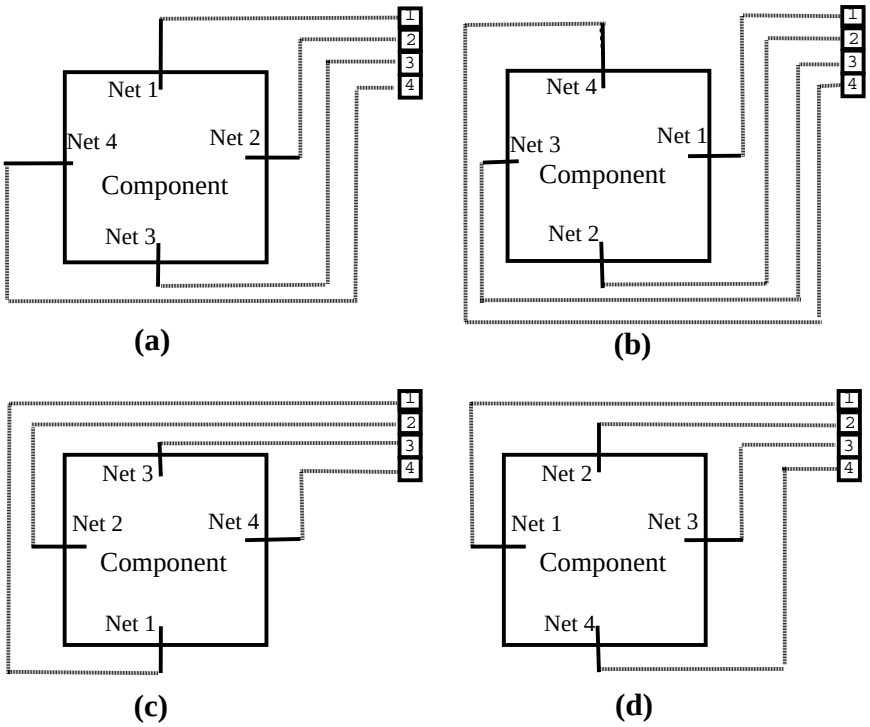


Figure 2.4: Potential solutions of 4-side escape.

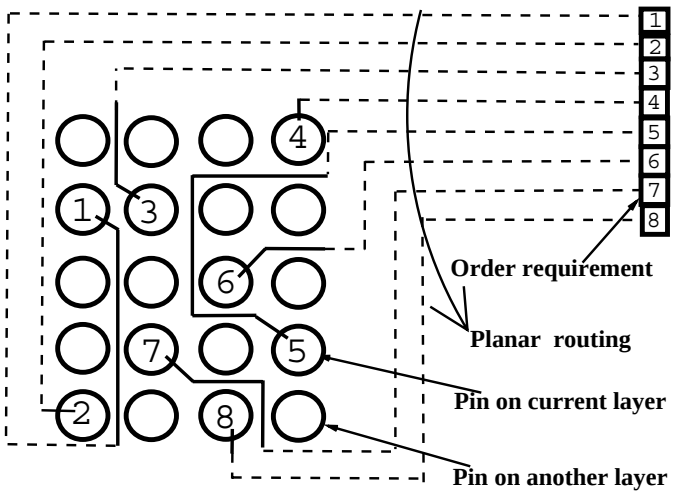


Figure 2.5: Ordered escape problem with cyclic escape ordering.

routing and only focus on the exiting points of these nets.) Obviously all of them are legitimate solutions, satisfying the escape ordering. However, only Fig. 2.4(a) is taken as legitimate by the SAT formulation in Section 2.2.1, because it is the only one satisfying the linear ordering. Since there are so many potential escape patterns missing, this deficiency greatly impairs the quality of SAT router.

In this section, we propose a way to formulate the cyclic ordering. Fig. 2.5 shows an example escape problem with cyclic ordering. The main idea is to modify the definition of slot variable so that Formula 2.11 still holds for cyclic ordering. To be detailed, redefine a slot variable as the concatenation of a flag bit and a net-index vector. The net-index part is exactly the original slot variable of Section 2.2.1. The flag bit is the most significant bit of the new slot variable, whose value is only determined by the exiting location of net 1. To explain the meaning of the flag bit, consider a specific situation where net 1 escapes at S_i ($i > 1$), which means the net-index part of s_i is 0. (To make effective use of memory, net 1 is indexed with 0 in implementation.) Apparently the net-index part of s_{i-1} must be greater than 0, because S_{i-1} can only be used by net 2, net 3, etc. each of which has index value greater than 0. Therefore, in order to satisfy Formula 2.11, or to be more specific, $s_{i-1} \leq s_i$, the flag bit of s_{i-1} must be 0, while the flag bit of s_i must be 1. Then the flag bits of all the slots within the range of $[\mathrm{S}_1, \mathrm{S}_{i-1})$, should be 0 to satisfy the constraint $s_1 \leq s_2 \leq \cdots \leq s_{i-1}$. Similarly, all the slots within the range of $(\mathrm{S}_i, \mathrm{S}_n]$ should have flag bits equal to 1. To sum up, the flag

bit of a slot is 1 if it is within the range of [S_i, S_n]; 0 otherwise. We can see that if net 1 escapes at S_1, then all the flag bits will be 1 and the new slot variable is actually equivalent to the original one in this case.

To formulate cyclic ordering, besides using Formula 2.11, we need to add another constraint formula F'_S for slot variables. F'_S is the conjunction of three types of formulae:

$$F'_S = F_\alpha F_\beta F_\gamma \tag{2.16}$$

F_α is the boundary constraint for flag bits: If a slot is the slot where net 1 exits the component; i.e., the net-index part of the slot is equal to 0, then the flag bit of this slot should be 1 and the flag bit of the slot in front of it should be 0.

$$\begin{aligned} &\forall s_i = (f_i, N_i),\ i > 1 \\ &N_i = 0 \Rightarrow f_i \overline{f}_{i-1} \end{aligned} \tag{2.17}$$

where f_i is the flag bit of s_i and N_i is a Boolean vector, the net-index part of s_i. Assume it is a k bit vector; then $N_i = (N_{ik}, N_{i(k-1)}, \cdots, N_{i1})$. It is well known that logical implication "$a \Rightarrow b$" is equivalent to the disjunctive operation "$\overline{a} + b$". Thus we can get the CNF expression for Formula 2.17 as follows:

$$\begin{aligned} F_\alpha &= \prod_{i=2}^{n} (N_i = 0 \Rightarrow f_i \overline{f}_{i-1}) \\ &= \prod_{i=2}^{n} (N_i \neq 0 + f_i \overline{f}_{i-1}) \\ &= \prod_{i=2}^{n} ((N_i \neq 0 + f_i)(N_i \neq 0 + \overline{f}_{i-1})) \\ &= \prod_{i=2}^{n} ((N_{ik} + N_{i(k-1)} + \cdots + N_{i2} + N_{i1} + f_i) \\ &\quad (N_{ik} + N_{i(k-1)} + \cdots + N_{i2} + N_{i1} + \overline{f}_{i-1})) \end{aligned} \tag{2.18}$$

F_β is the constraint for consistent flag bits: if a slot S_i has a flag bit equal to 1, then all the slots within the range of (S_i, S_n] should also have flag bits

equal to 1.

$$\begin{aligned} &\forall i,\ 1 \leq i < n \\ &f_i = 1 \Rightarrow \forall j,\ i < j \leq n,\ f_j = 1 \end{aligned} \tag{2.19}$$

By replacing the implication expression, we get the CNF formula as follows:

$$\begin{aligned} F_\beta &= \prod_{i=1}^{n-1} \prod_{j=i+1}^{n} (f_i = 1 \Rightarrow f_j = 1) \\ &= \prod_{i=1}^{n-1} \prod_{j=i+1}^{n} (\overline{f}_i + f_j) \end{aligned} \tag{2.20}$$

F_γ is the boundary constraint for the net-index parts of the slot variables. For the convenience of explanation, as before, we suppose net 1 escapes at S_i, where $i > 1$. (Note that $i > 1$ means $f_1 = 0$.) Then the escape ordering requires that the net-index part should keep increasing, when we traverse the slots in a clockwise order, from S_i to S_{i+1}, $\cdots$, S_n, S_1, $\cdots$ until S_{i-1}.

$$N_i \leq N_{i+1} \leq \cdots \leq N_n \leq N_1 \leq \cdots \leq N_{i-1} \tag{2.21}$$

$N_i \leq N_{i+1} \leq \cdots \leq N_n$ is guaranteed by $s_i \leq s_{i+1} \leq \cdots \leq s_n$ through F_S. This is because in the range of $[\mathrm{S}_i,\ \mathrm{S}_n]$, all the flag bits are equal to 1 and the relative magnitude of a slot variable is only determined by the net-index part. Similarly, $N_1 \leq \cdots \leq N_{i-1}$ is also guaranteed through F_S. The only thing left is to make sure $N_n \leq N_1$. F_γ is created for this purpose.

$$\begin{aligned} F_\gamma &= \overline{f}_1 \Rightarrow N_n \leq N_1 \\ &= f_1 + (N_n \leq N_1) \end{aligned} \tag{2.22}$$

We can use the methods presented in Section 2.2.2 to convert Formula 2.22 into a CNF expression. Now the original constraint formula for escape ordering F_S is replaced by the following formula:

$$F_S F'_S = F_S F_\alpha F_\beta F_\gamma \tag{2.23}$$

Now we need to prove that $F_S F'_S$ is the correct constraint for (cyclic)

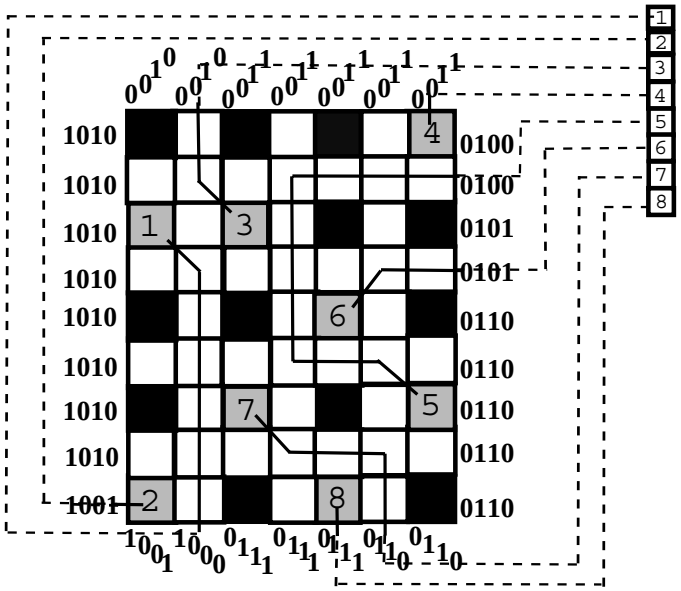


Figure 2.6: Escape solution to the escape problem in Fig. 2.5. Only slot variable assignments are shown.

escape ordering. (We only focus on the ordering in the proof, because it is the only difference between the new formulation and the one in Section 2.2.1.) We prove it in two directions. Firstly, we prove every variable assignment satisfying $F_S F'_S$ is corresponding to a legitimate escape ordering. It is easy to see that by satisfying $F_\alpha F_\beta$, we make sure that the flag bits of all the slots within the range of [S_1, S_{i-1}] are 0 (suppose S_i is the escaping slot of net 1), while the flag bits of the slots within the range of [S_i, S_n] are 1. Then by satisfying F_S and F_γ, we know that the net indices on the slots increase clockwise starting from S_i; i.e., the escape ordering is correct. Secondly, we need to prove that for every legitimate escape solution, we can find an assignment of variables which satisfies $F_S F'_S$. Suppose in the escape solution, net 1 escapes through S_i; then we set the flag bits of S_i, S_{i+1}, $\cdots$, S_n to be 1 and the flag bits of S_1, S_2, $\cdots$, S_{i-1} to be 0. This assignment satisfies $F_\alpha F_\beta$. The net-index parts of these slot variables can be assigned in a similar way as in Section 2.2.1; i.e., the net-index part of the escaping slot of net 1 is set to 0, the net-index part of the escaping slot of net 2 is set to 1, the net-index parts of all the slots in between are also set to 1, and so on. Since this is a legitimate solution, it is not hard to prove $F_\gamma F_S$ is also satisfied by this assignment. Fig. 2.6 shows the values of slot variables (the leftmost bit is the flag bit) for the escape pattern in Fig. 2.5.

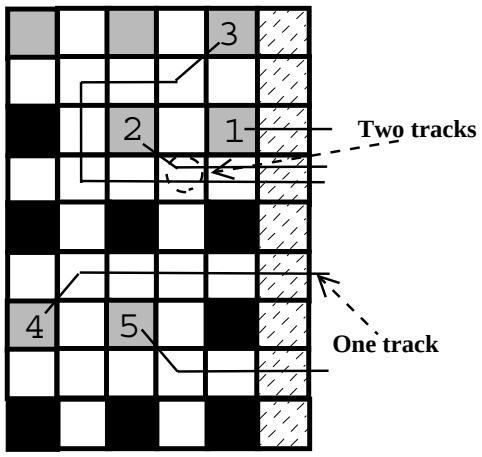


Figure 2.7: 2-track solution.

2.3 Handling Infeasible Problems

Usually there is only one track used in each grid cell, since one track is the best for signal integrity. However, in some occasions, when no escape solution exists by using only one track, the design rule also tolerates another track in local regions, as shown in Fig. 2.7. The SAT solution not only judges whether a one-track solution exists, but also gives a clue to add additional tracks when necessary. This judgment and dynamic track addition ability was never considered by the monotonic router [22] or the ILP method [23]. Moreover, in the field of SAT-based physical design algorithms, this is also the first work to take advantage of the unsatisfiable result.

A typical SAT solver works in this way: by continuously trying variable assignments and learning from the failed assignments, the solver finds more and more necessary variable values for satisfying the Boolean function. There are two ending conditions for this search process: one is successfully finding a set of variable assignments, which make the function equal to 1; the other is finding a conflict clause, which can never be true based on the necessary variable values. In the second case, the conflict clause gives us hints about the cause of the failed escape, or the regions where additional tracks are needed.

Define the variables in the conflict clause to be conflict variables. Based on the types of conflict variables, we have different track addition strategies.

Hint 1 A node variable is a conflict variable. In this case, the most possible reason for the failure of routing is that several nets are competing

for this node. Once this node is allocated to one net, the routing of the other nets cannot be finished and conflict happens. The solution is to add an additional track within this node. For Fig. 2.7, the variable corresponding to the highlighted node is a conflict variable.

Hint 2 An edge variable, which is attached to a pin, is a conflict variable. In this case, the possible reason for the failure of routing is that the net is routing away from the pin in a wrong direction. If the variable has a positive sign in the conflict clause, then the correct direction to route the net away from the pin should be along this edge. This is obvious, since using this edge means the edge variable is equal to 1, and then the conflict clause is equal to 1. In contrast, if the variable has a negative sign, then the correct direction should be any direction but the one along that edge. The router (actually the SAT solver, since we have transformed the routing problem into a SAT problem) failed to use the correct direction in the first place, because there was not enough routing resource there. Hence, the solution to this kind of conflict is adding tracks around the pin in the correct direction.

Note that usually a conflict clause has more than one variable, so it is possible that Hint 1 and Hint 2 will be used together to resolve conflicts.

Hint 3 A slot variable is a conflict variable. In most of the cases, this kind of conflict should be handled in the same way as Hint 1. However, if the conflict clause is composed of only slot variables, this is a hint of ordering problem. This situation is more complex than the other two, since the ordering problem is a global problem, which cannot be easily solved by adding additional routing resource around the slots. Most often, a wrong ordering in slots is caused by choosing wrong directions when the nets route away from the pins. Thus the solution is to trace back to the pins of the conflict nets and add more routing tracks around these pins. Hopefully with the additional tracks, the router will find a way to route the nets away from the pins in correct directions.

2.4 Handling Large Problems

Although our SAT based router can handle mid-sized problems in reasonable CPU time, its running time grows exponentially with the problem size. Therefore, it is not realistic to directly apply the SAT router to large-scale

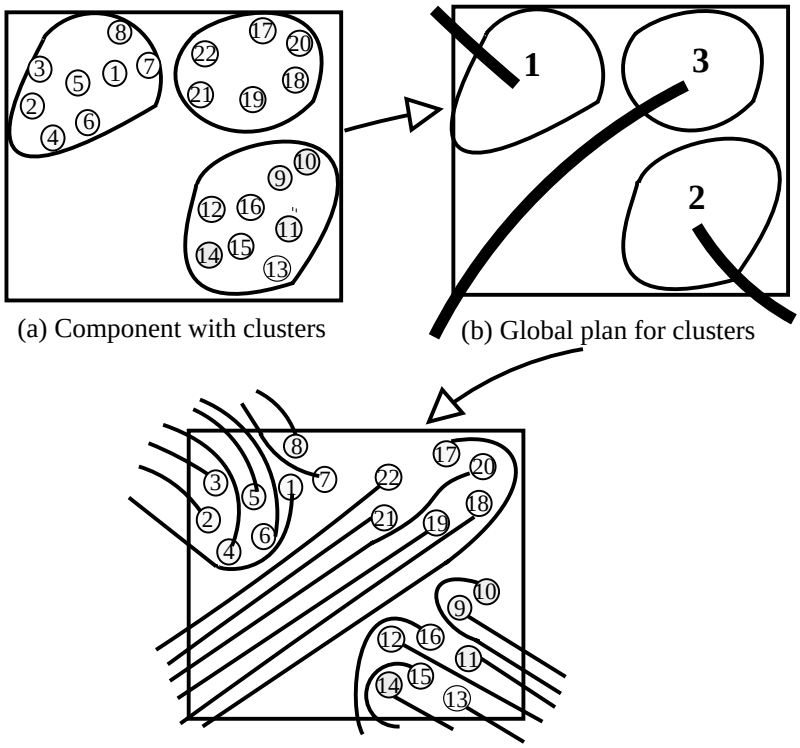
problems. In this section, we first present a strategy to apply the SAT router efficiently for large-scale problems with clustering structure. Furthermore, the SAT router is used as a general rip-up and reroute technique to fix the escape solution obtained by any heuristic router.

2.4.1 SAT for Clustering-Structure Problems

We observe from industrial benchmarks that a great portion of large-scale escape problems have a clear clustering structure; i.e., pins can be grouped into several disjoint clusters. This clustering structure comes from the bus routing problem in real world design. Since the nets belonging to one bus are always routed side by side, their end pins can be clustered together. Therefore, instead of escaping the whole component altogether using our SAT approach, which is time-consuming and may not even terminate due to the large problem size, we can escape each cluster separately using SAT router. The main idea is illustrated in Fig. 2.8. (Note that Fig. 2.8 only shows the routing topology, not the exact routing result.)

This divide-and-conquer strategy works due to the following reasons. First, since each cluster is composed of neighboring pins with neighboring indices, it is reasonable to take it as an isolated escape problem. Second, pins in each cluster are distributed closely with each other. Hence, on one side, the escape pattern within each cluster may be a very complicated one and any heuristic based method can hardly get a solution; on the other side, each cluster is a small-size problem so that the SAT solver can work quite efficiently. Finally, compared with intra-cluster escape, to complete the routing from cluster boundaries to the component boundary is a much easier planar routing problem.

Since the number of clusters is usually quite small, the global plan can be done in many different ways such as enumerating all the possible combinations of escape directions, or global routing using maze router or any other available routers, or even manual routing. What is interesting is that we can also convert global planning into another escape problem and take advantage of SAT router to search for multiple solutions. The main idea is to build a coarse routing grid, project clusters to the pins on the coarse grid, and use SAT router to get multiple escape patterns for clusters. The reason



(a) Component with clusters

(b) Global plan for clusters

(c) Detailed escape (Inner-cluster escape finished by SAT)

Figure 2.8: Divide-and-conquer for component with clustering structure.

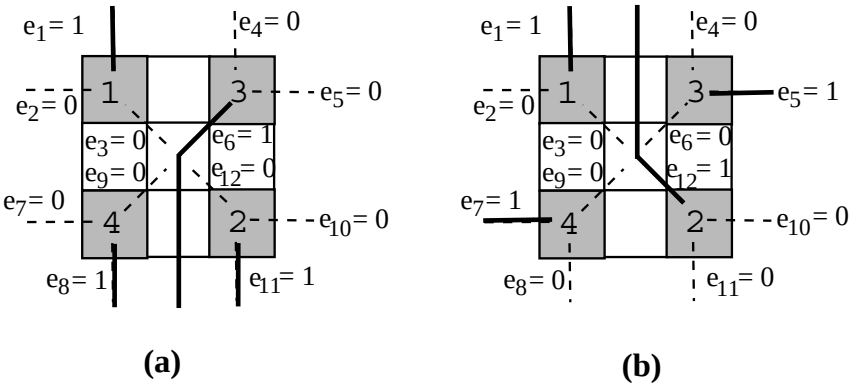


Figure 2.9: Solutions with different escape directions.

why we need multiple patterns is that the escape direction of each cluster directly determines the escape sides of pins within the cluster. (For example, as shown in Fig. 2.8, the first cluster escapes to the top left corner, which determines the first eight pins can only escape on the top and left sides of the cluster.) Since not every escape direction can lead to successful inner-cluster escape, we need to explore several possible directions before finally finding the correct one. We extend the original SAT router as follows to get multiple escape solutions.

We know from Section 2.2 that each pin is connected with several edges and a legitimate escape requires that one and only one of these edge variables is 1, which is corresponding to the escape direction of the pin. Therefore, to get different escape directions means getting different values for pin-connected edge variables. For example, Fig. 2.9 shows the values of all the pin-connected edge variables in two different escape solutions. The way to get multiple solutions is to run SAT router repeatedly with a new solution each time. To avoid the appearance of duplicate patterns, a constraint is added to the SAT formulation each time we get a new solution. For example, after getting the solution in Fig. 2.9(a), we add the following constraint formula:

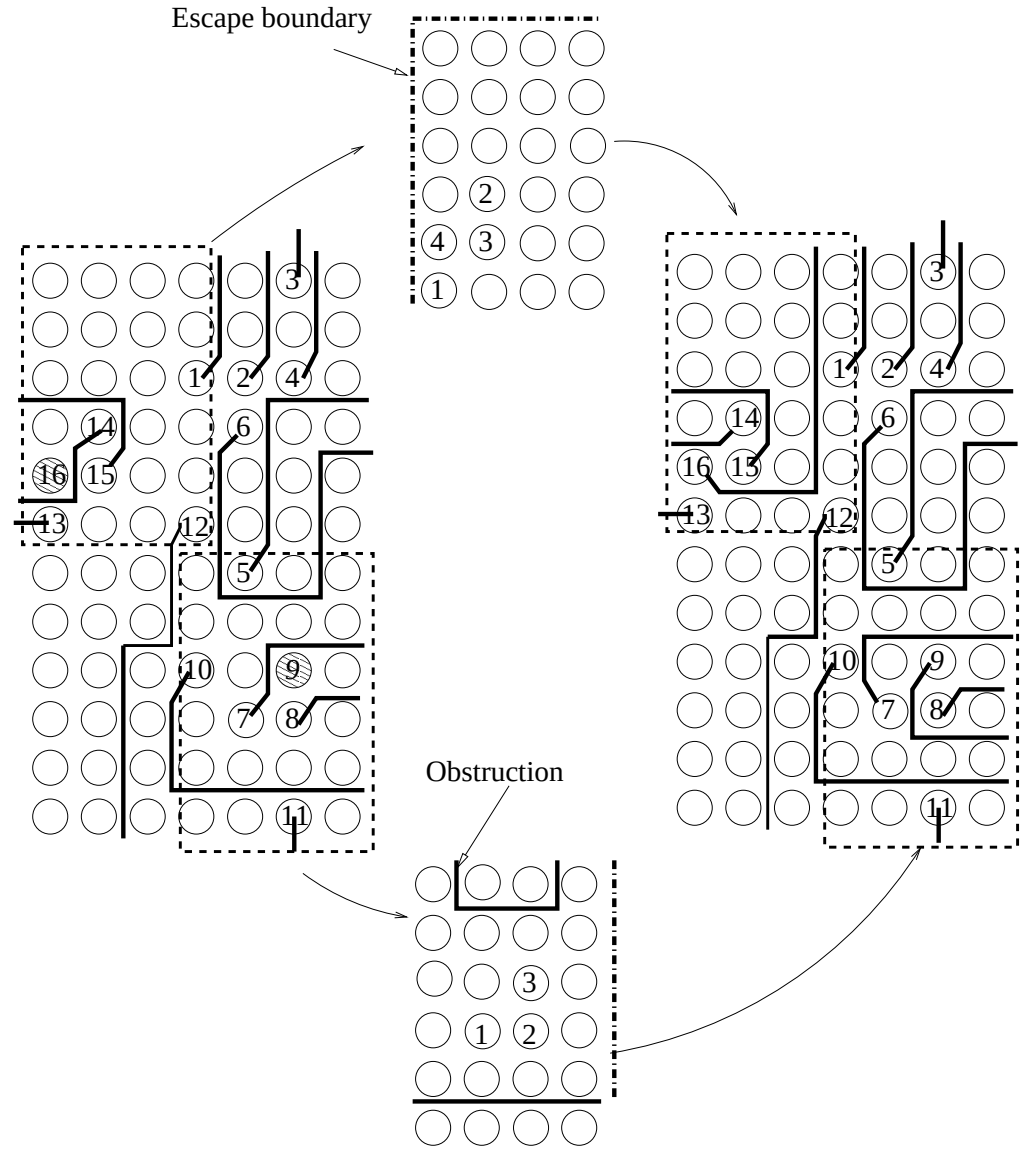
$$(\overline{e}_1 + \overline{e}_6 + \overline{e}_8 + \overline{e}_{11}) \tag{2.24}$$

To make Formula 2.24 equal to 1 means that we cannot have e_1, e_6, e_8 and e_{11} equal to 1 at the same time; i.e., it prevents the solution in Fig. 2.9(a) appearing again.

We will see in Section 2.5 that by taking advantage of the clustering structure, this divide-and-conquer strategy makes the SAT router applicable to large-scale problems without compromising routability at all.

2.4.2 SAT for Rip-up and Reroute

From a more general view, SAT router can also be used as a rip-up and reroute technique to fix the escape solution obtained by any other router. This is useful for the case where the divide-and-conquer strategy does not work because the clustering structure is not very clear over the whole component, or the scale of a single cluster is still too large for SAT. In such case, we can finish the escape in two steps: initial routing and rip-up and



(a) Initial escape (2 nets unrouted) (b) Fixed solution

Figure 2.10: Fixing unrouted regions by SAT.

Table 2.1: Experimental Results on moderate cases.

	#Row	#Col	#Pin	#Slot	#Var	#Clause	Run time (s)
Ex1	13	4	16	25	288	1782	0.02
Ex2	11	5	25	39	420	3369	0.94
Ex3	10	6	25	41	475	3509	0.14
Ex4	10	10	15	76	684	3214	6.93

reroute. First, we use some heuristic algorithms to get an initial solution very quickly. If there are some unrouted nets (they exist most of the time, owing to the limitation of heuristics in exploring complex escape), we zoom in on the unrouted regions and make use of SAT router to complete the escape. An unrouted region should include one or more unrouted nets, some neighboring nets, which are ripped up to provide space for rerouting, and some fixed routing of other nets, which becomes the obstruction for the local escape problem. It is very easy for SAT router to handle obstruction, because we can simply get rid of a node/edge from the graph if it is blocked.

We use an example to illustrate this two-step escape idea. Fig.2.10(a) is an initial routing obtained from a heuristic method, where two nets, net 9 and net 16, are unrouted. We zoom in on the two unrouted regions (enclosed by the dash rectangles), and use SAT router to exactly resolve both problems. Finally we get a 100% routed solution in Fig.2.10(b).

2.5 Experimental Results

We implemented the SAT based escape routing algorithm in C++. The SAT router includes two modules. One is the pure SAT router introduced in Section 2.2. If the escape pattern does not exist with current routing resource, the second module adds additional tracks in resource-insufficient area and then calls the first module again. This process iterates until a solution is found, or any more tracks are unacceptable.

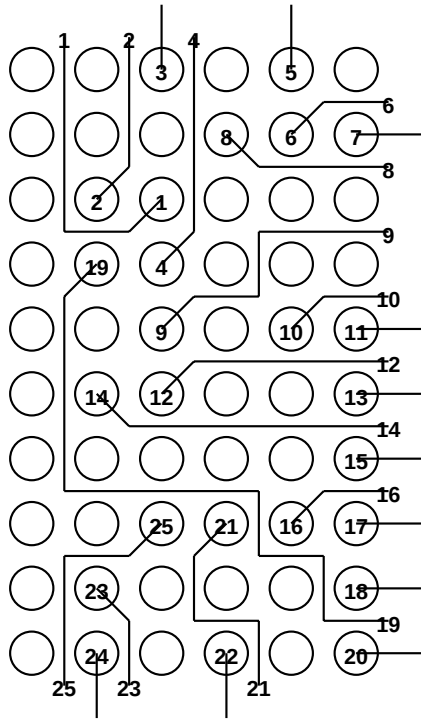


Figure 2.11: Escape result for Ex3.

We tested the SAT router on a Pentium 4 2.8 GHz system with 4 GB memory and a Unix operating system. We use MiniSat [33] as the SAT solver. We tested three escape problems of real board design from industry and one artificial case (Ex3), which has an even more complex escape pattern than industrial ones. The characteristics of these cases as well as the experimental results are shown in Table 2.1. Each case has a #Row×#Col pin array as input. #Pin is the number of pin terminals which need to escape on the current layer. #Slot is number of slots which could be used for escape. These slots can be distributed on all four sides of the input component. #Var and #Clause are the numbers of variables and clauses in the SAT formulation. Ex2 has no solution under one track constraint, so additional tracks were added based on the technique introduced in Section 2.3. The escape result for the artificial case is shown in Fig. 2.11.

Extensive experiments show that for the pure SAT router, the maximum scale it can handle in acceptable time (within ten minutes for us) is approximately 10×10 pin array and the number of nets does not affect the run time as much as the size of the pin array.

The following experiments are performed to test our strategies of using

Table 2.2: Experiments on clustering-structure benchmarks

	#Row	#Col	#Pin	#Slot	#Cluster	CPU (s)
Ex5	14	13	15	104	3	0.01
Ex6	20	8	20	54	2	0.02
Ex7	20	20	53	156	6	9.16

Table 2.3: Experiments for rip-up and reroute

	#Row	#Col	#Pin	#Slot	Heuristic Router		Heuristic + SAT	
					#Routed	CPU (s)	#Routed	CPU (s)
Ex8	8	13	16	80	10	0.03	16	0.61
Ex9	20	6	32	100	31	0.01	32	0.02
Ex10	20	20	58	156	49	0.31	58	9.43

the SAT router on large scale problems. We first test the divide-and-conquer strategy introduced in Section 2.4.1 on three escape problems Ex5, Ex6 and Ex7. Their characteristics are shown in Table 2.2. As in Table 2.1, #Pin represents the number of pin terminals which need to escape on current layer, and these pins are grouped into #Cluster clusters. Ex5 and Ex6 are real board designs from industry. Ex7 is an artificial case, which is created by imitating the feature of industrial benchmarks. All these cases are 100% routed and the running time is shown in the last column of Table 2.2. Fig. 2.12 shows the escape solution of Ex7, where each cluster is highlighted by a dashed rectangle.

We use the other three cases, Ex8, Ex9 and Ex10, to test the rip-up and reroute technique presented in Section 2.4.2. None of these benchmarks has clear clustering structure. Table 2.3 lists the characteristics of these three cases. Ex8 and Ex9 are industrial benchmarks and Ex10 is an artificial one derived according to the characteristics of real board design. We first use a heuristic algorithm to get an initial solution and list the number of routed nets (#Routed) and CPU time in Table 2.3. Since none of these cases are completely routed by the heuristic method, we then use SAT router to fix the unrouted regions. The last two columns of Table 2.3 list the number of routed nets after rip-up and reroute and the total CPU time (including the running time of both the heuristic router and SAT router). For comparison, we show the escape solutions of Ex10 before and after rip-up and reroute in Fig. 2.13 and Fig. 2.14 respectively. The shaded pins in Fig. 2.13 correspond to the unrouted nets. We use SAT router to fix the solution in two local regions, highlighted by dashed rectangles, and get a completely routed solution.

2.6 Conclusion

A SAT based ordered escape routing approach is proposed in this chapter. For the moderate cases from industry, it guarantees to find the solutions in reasonable CPU time. It can also handle the infeasible cases by dynamically adding tracks. Then we proposed two approaches to extend SAT for large-scale problems: a divide-and-conquer strategy to handle problems with clustering structure, and a rip-up and reroute technique to fix the routing solution obtained by any router. Experimental results show that these ap-

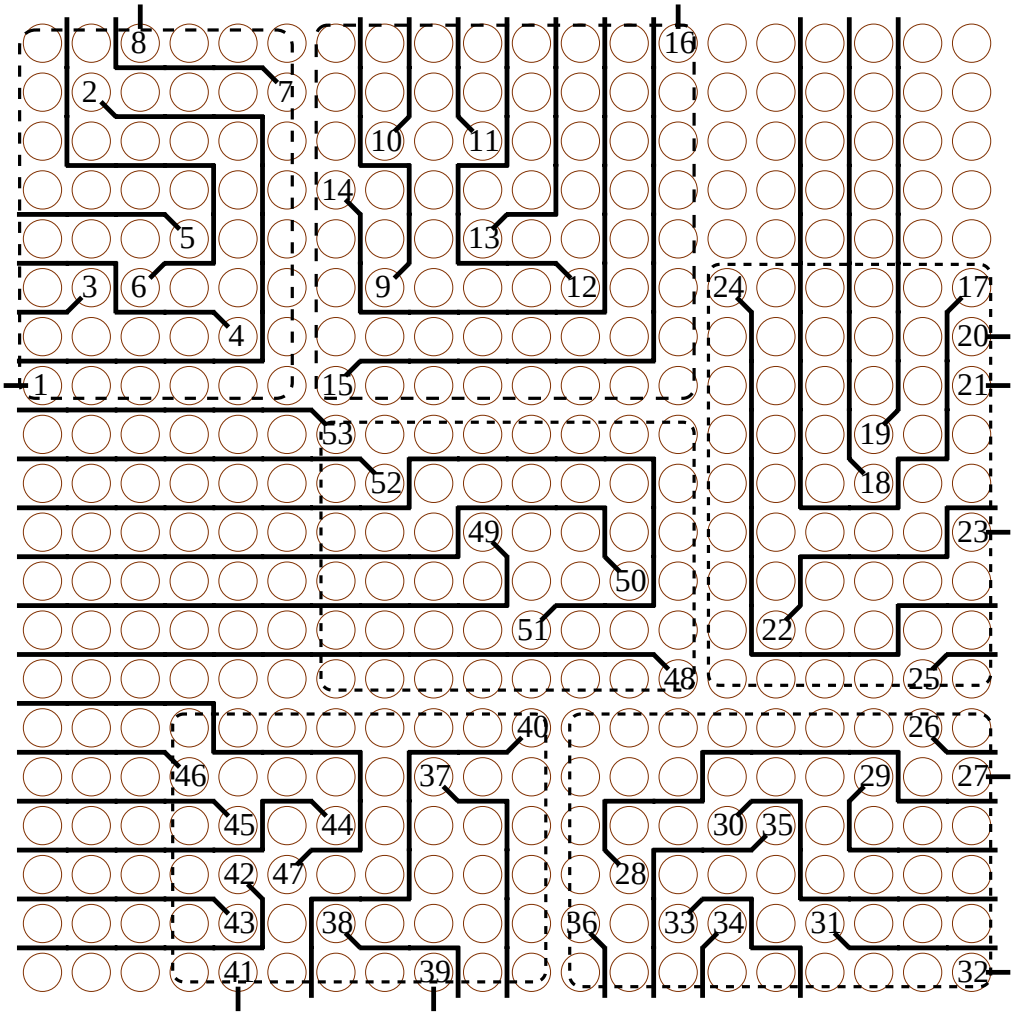


Figure 2.12: Escape solution of Ex7.

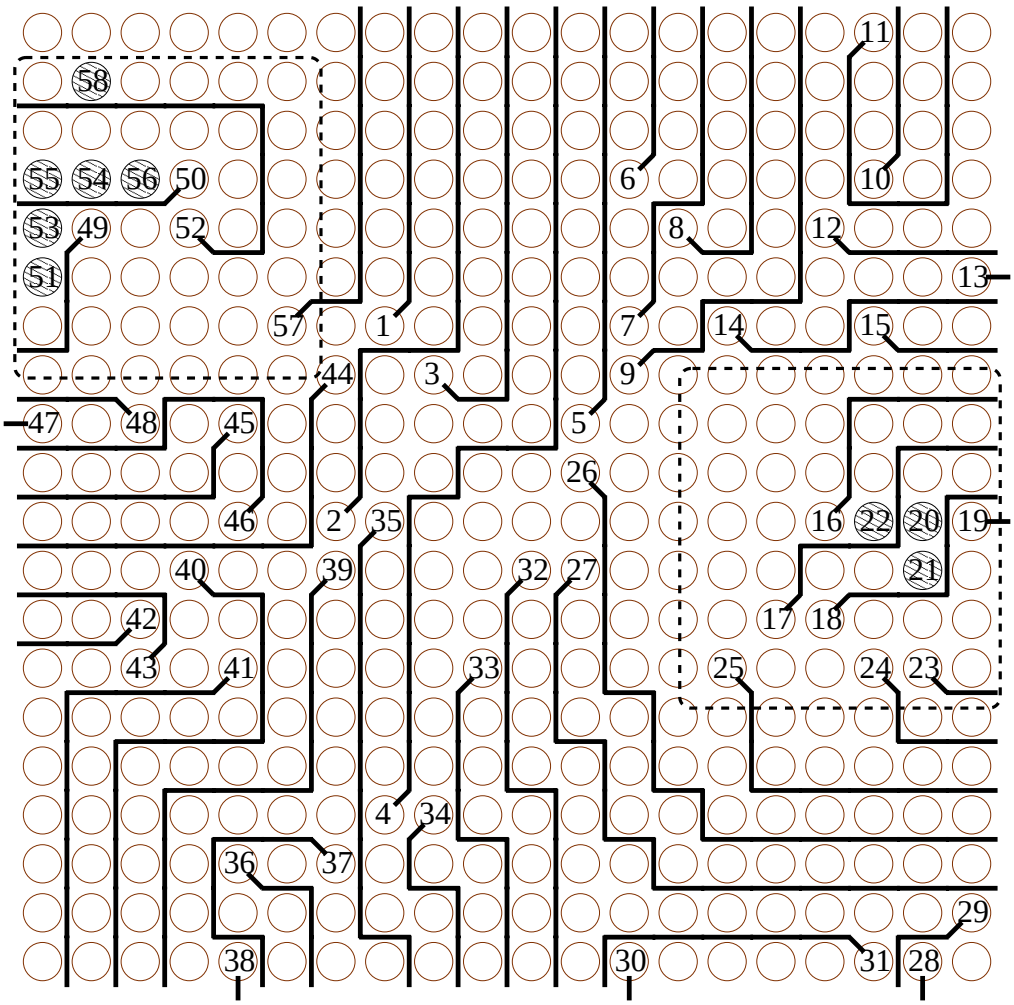


Figure 2.13: Escape solution of Ex10 obtained by the heuristic algorithm.

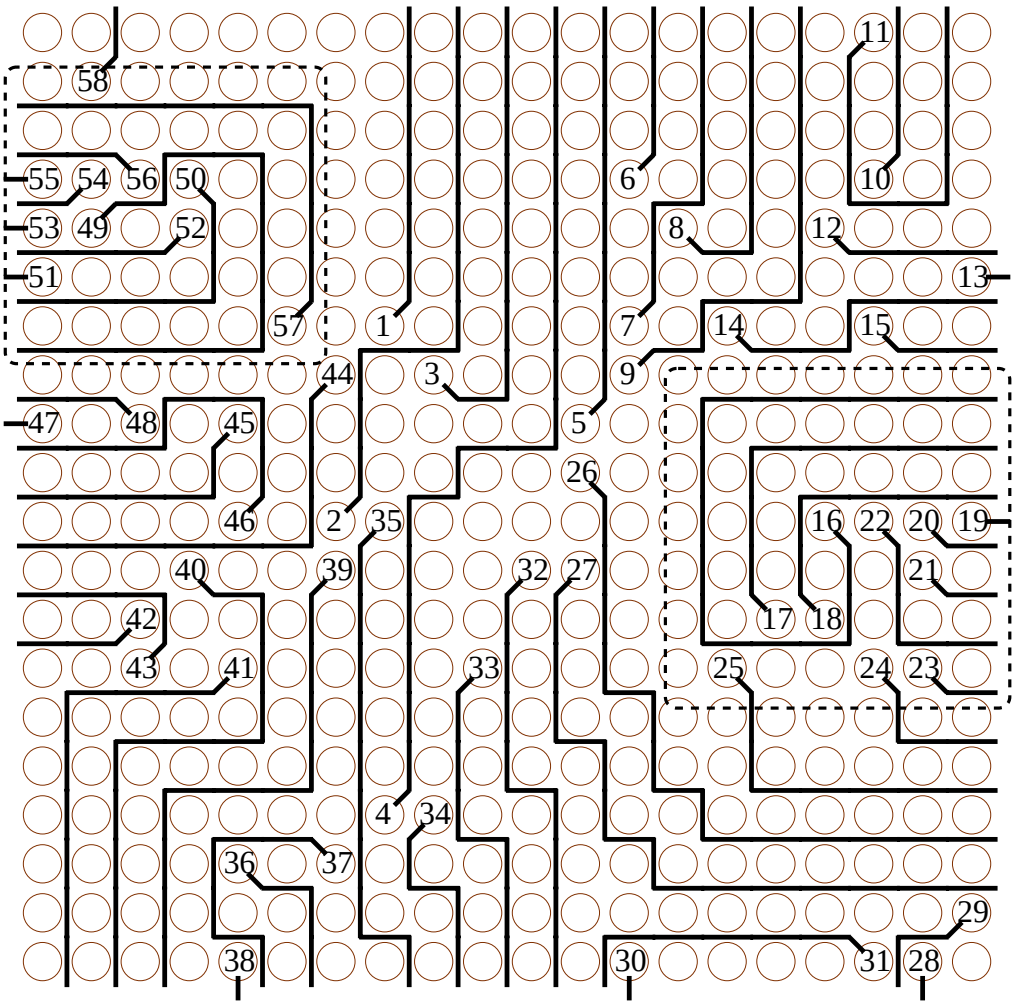


Figure 2.14: Escape solution of Ex10 fixed by SAT.

proaches perform very well on industrial benchmarks.

CHAPTER 3

BOUNDARY ROUTING FOR SIMULTANEOUS ESCAPE

3.1 Introduction

The SAT router introduced in Chapter 2 only applies to a single component. In real PCB routing, however, we need to determine the routing between two different components. In order to guarantee a consistent net ordering between the two connected components, one can run an unordered escape on the first component and then our SAT router on the second component matching the net ordering from the first component. This solution only works well if the ordering we get from the first component happens to be a suitable ordering for the second one. A better way is to simultaneously escape from both components, and this simultaneous 2-component escape routing problem is the subject of this chapter. Figure 3.1 illustrates the simultaneous escape problem where we are given pins connecting two components and we are asked to route all pins to their respective component boundaries such that inter-component connections are planar, i.e., no net crossing. The inter-component connections are shown in dashed lines and their detailed routing paths will be determined in a subsequent area routing phase. Existing published algorithms [34, 35] for this problem are based on pattern routing where each pin is given a small number of fixed escape patterns (e.g., L-shaped routes escaping no more than two tracks above/below the pin). These algorithms would perform well if the pins on the two components are reasonably well aligned. However, more complicated escape problems cannot be solved by escape algorithms based on fixed/limited escape patterns. For example, the routing solution in Figure 3.1 cannot be captured by pattern routing.

In this chapter, we present a new simultaneous escape routing algorithm,

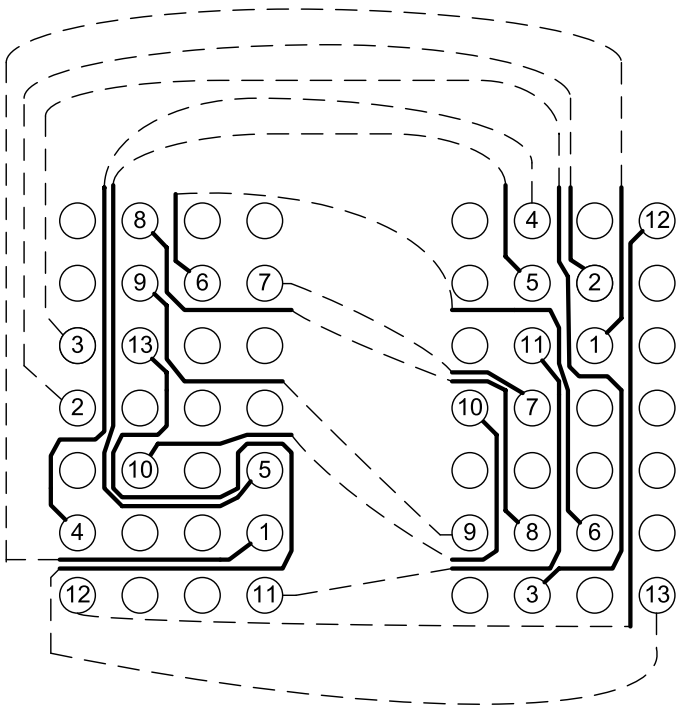


Figure 3.1: Simultaneous escape routing.

B-Escape, which is based on a novel boundary routing approach. B-Escape can solve complicated escape problems in a very short time. We tested our algorithm on a set of industrial escape problems. These problems were previously successfully routed by experienced layout experts taking about 8 hours per problem. B-Escape successfully solved all of them while Cadence Allegro PCB router was only able to complete the routing of half of the problems. In addition, we propose a clustering strategy targeting at large escape routing problems. Experimental results show that this clustering strategy can significantly cut down the runtime of our router when solving large problems.

The rest of the chapter is organized as follows: Section 3.2 introduces our boundary routing approach. Section 3.3 introduces the strategy to decide net ordering for the simultaneous escape routing problem. We limit our discussion to 1-side escape, because as shown in Figure 3.2, 4-side escape can always be transformed into 1-side escape by adding more rows or columns. Also, to better present our ideas, we use a grid structure in Section 3.2 and 3.3. Then we will discuss some special implementation issues on circuit boards in Section 3.4. Section 3.5 presents our clustering strategy for solving

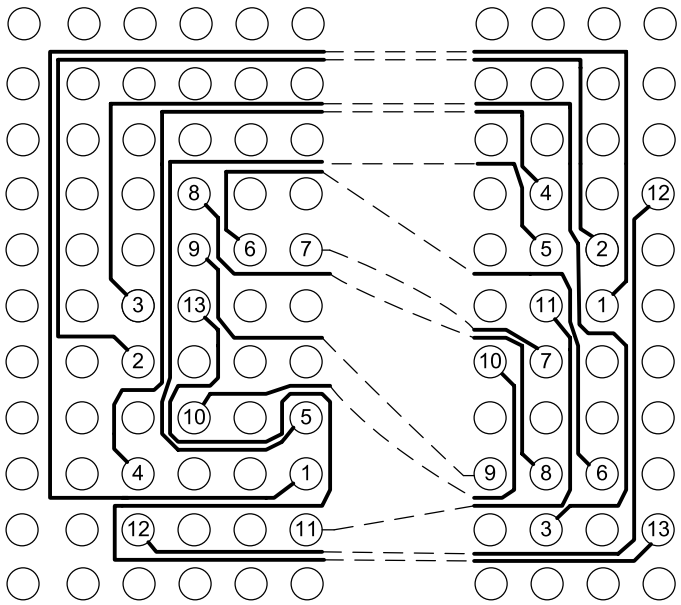


Figure 3.2: Conversion from 4-side escape (Figure 3.1) to 1-side escape.

large problems. Experimental results are shown in Section 3.6. Finally, Section 3.7 concludes this chapter.

3.2 Boundary Routing

Before we present our simulteneous escape routing algorithm, we first introduce the foundation of our algorithm: boundary routing. To better present the idea of boundary routing, we assume that we are given a rectangular routing area, which we call the *component*. We also assume that there is a grid structure inside the component, like in Figure 3.3. (In later figures, the underlying grid is hidden for conciseness of illustration.) Selected grid points, which we call *pins*, are labeled $1, 2, \ldots, n$, and are expected to be routed to the right side of the component. The ordering of the routes must be increasing along the right side from top to bottom. We call such a routing problem the *fixed-ordering escape problem*. In the following, we will present the boundary routing methodology which is very effective on such fixed ordering escape problems.

We define the *routing boundary* as the boundary of the maximum routable region for the unrouted pins. Initially, the routing boundary is just the boundary of the component. After we route a pin, the boundary needs to be

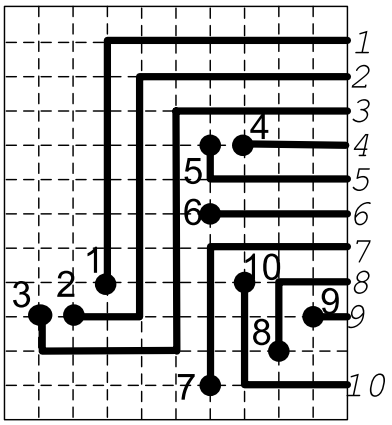


Figure 3.3: Fixed-order escape problem.

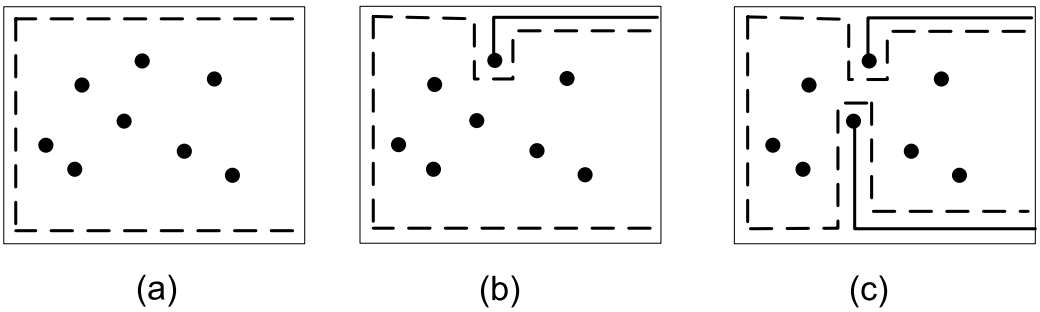


Figure 3.4: Routing boundary.

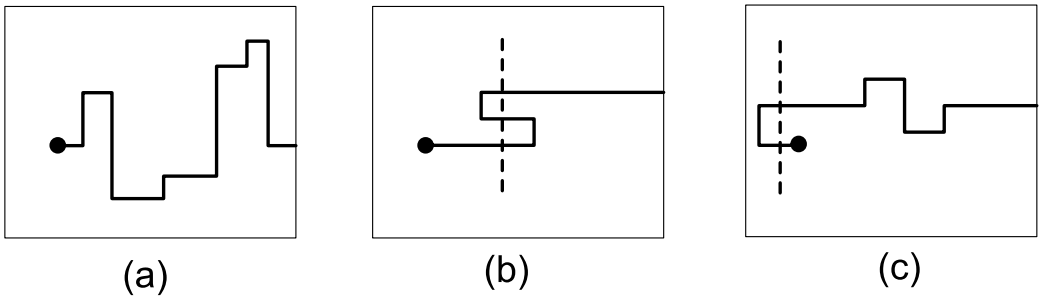


Figure 3.5: Monotonic routing (a) and non-monotonic routing (b), (c).

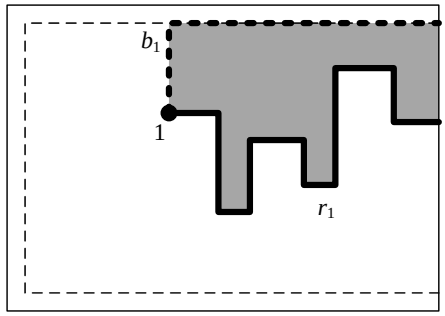


Figure 3.6: Illustration for the proof of Theorem 1. The gray region should contain no pin or route.

shrunk to exclude the routing of that pin. See Figure 3.4 for an illustration of the routing boundary (shown as a dashed line).

We can make one observation from Figure 3.4: when we route a pin, we will make more space available for the remaining pins if we follow the routing boundary as much as possible. This observation leads to our boundary routing strategy: whenever we route a pin, we first route it to the routing boundary and then follow the boundary. Such a simple strategy turns out to be very powerful in solving fixed-ordering escape problems. The following statements show that the strategy is optimal for a certain type of escape problem.

Definition 1. *A routing path is said to be* monotonic *if the intersection of any vertical line with the routing path is either empty, or a single point or a single line segment.*

In Figure 3.5, (a) shows a monotonic routing path while (b) and (c) give two non-monotonic routing paths because we can find a vertical line that intersects the routing at two points.

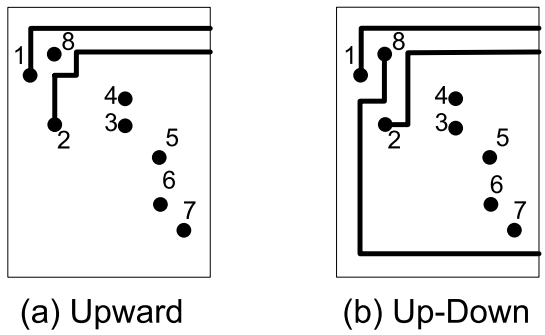


Figure 3.7: Up-down mode vs. upward mode.

Theorem 1. *If all routing paths in a fixed-ordering escape routing solution are monotonic, then this routing solution can be captured by the boundary routing strategy.*

Sketch. Consider pin 1 with route r_1 in the solution (see Figure 3.6). We draw a path from pin 1 straight up to the upper boundary of the component and then follow the upper boundary to the right side of the component. We call this path b_1. The region between b_1 and r_1 should contain no other pin or routing segment. This is because such pin or segment has a label larger than 1 and therefore a destination below r_1. Then, the monotonic routing from the pin or segment must intersect r_1 in order to reach that destination, causing a conflict.

Since the region contains no other pin or routing, we can replace r_1 with b_1 without changing the topology of the routing solution. By repeating such a procedure for each pin, we can transform a solution into a boundary-routing solution. This means that we can obtain a solution with the same topology through the boundary routing strategy. □

The ability of boundary routing is not limited to monotonic solutions. By varying the way we route from the pin to the routing boundary and the direction we follow the boundary, we can derive various routing styles. We found six routing modes to be simple and effective in our experiments. The first mode is *upward* mode: we route the pin straight up until it meets the boundary and then follow the boundary clockwise. The proof of Theorem 1 shows that this mode captures the monotonic routing solution. We can also route the pin straight down to the boundary and follow the boundary in a counter-clockwise direction. We call this routing mode *downward* mode.

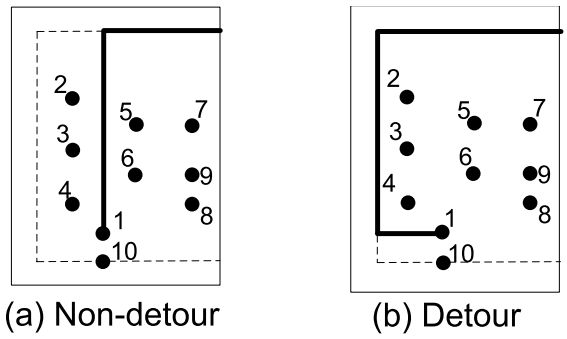


Figure 3.8: Detour vs. non-detour. The routing of Pin 1 in (a) blocks pins $2, 3, 4$. Leftward detour in (b) resolves this issue.

Obviously, when we use upward mode, we need to route the pins in increasing order of their labels, and for downward mode, we need to route the pins in decreasing order. Sometimes, pins may be trapped and become unroutable if we route the pins in pure increasing or decreasing order as shown in Figure 3.7 (a).

The *up-down* mode can resolve this issue by alternating between the upward mode and downward mode. In this mode, whenever we find that routing in upward mode will completely block another pin, we will switch to downward mode and route the unrouted pin with the largest label. In Figure 3.7, we found that routing pin 2 in upward mode completely blocks pin 8, so we switch to downward mode and route pin 8 instead. Since all the routing modes introduced so far route straight up or down to the routing boundary without detouring to the left, we call them *non-detour upward/downward/up-dowm* modes. Note that the detour of net 8 in Figure 3.7 (b) is generated by following the routing boundary, which does not conflict with the definition of non-detour mode.

Routing straight up or down to reach the boundary might not be the best choice for some cases. For example, routing a pin straight up may block other pins behind it and make those pins difficult to escape (see Figure 3.8).

Therefore, we also introduce the *detour* style, which is to route each pin leftward to reach the boundary and then follow the boundary. The detour style is combined with the upward mode, downward mode and up-down mode to form three new modes: *detour upward* mode (route each pin leftward to reach the boundary and then follow the boundary clockwise), *detour downward* (route each pin leftward to reach the boundary and then follow the

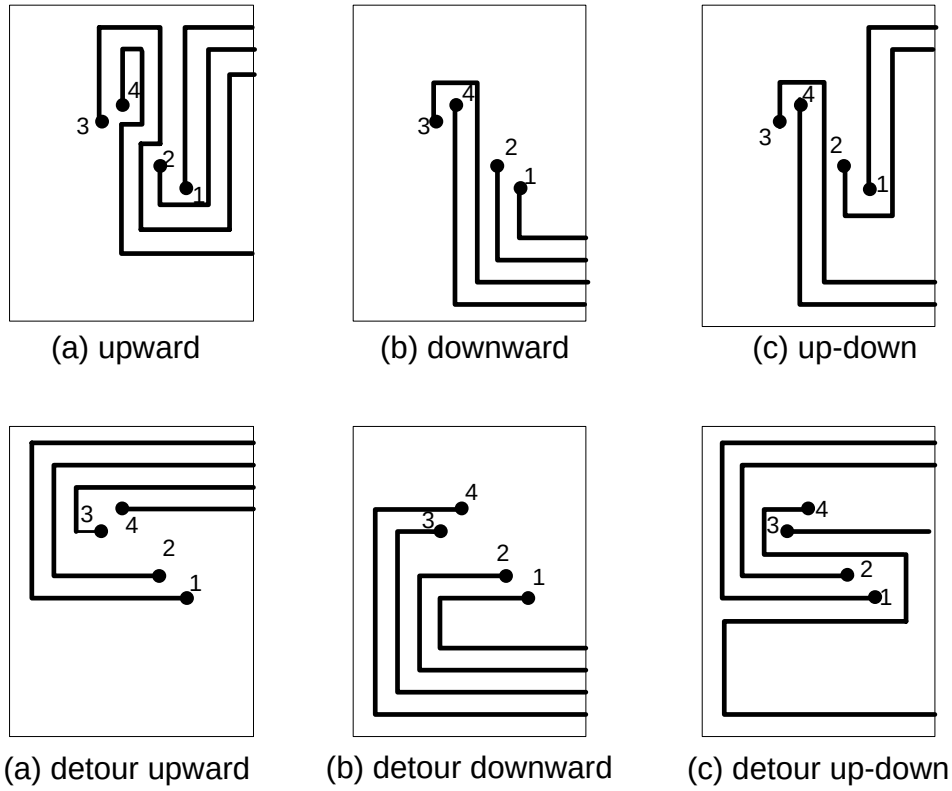


Figure 3.9: Different routing modes.

boundary counter-clockwise) and *detour up-down* mode (alternate between detour upward and detour downward). Figure 3.9 shows an example of routing the same problem using the six modes. Note that non-detour up-down mode and all the detour modes can capture non-monotonic routing solutions.

Although the boundary routing strategy and the six modes we use seem simple, they can cover a large number of problems we encounter in industrial data. Many routing solutions that seem complex can also be captured by one of the six modes. For example, the routing solution in Figure 3.10 seems very complex. However, the same topology can be captured by boundary routing under up-down mode.

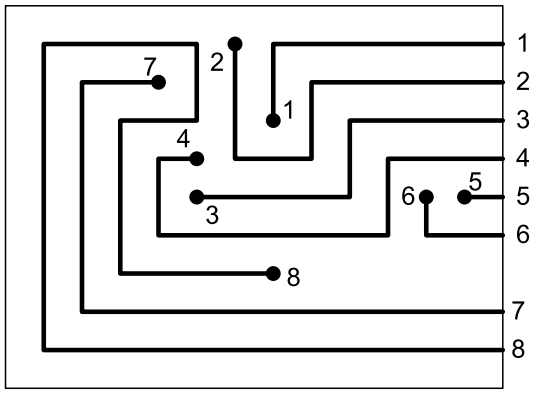


Figure 3.10: A routing solution that can be captured by up-down boundary routing mode.

3.3 Dynamic Net Ordering

Given two side-by-side components and a set of two-pin nets with one pin in the left component and the other pin in the right component, a *simultaneous escape* is to find escape solutions in both components so that they are honoring the same escape ordering. An example of a simultaneous escape solution is shown in Figure 3.11(a). If we already know the escape ordering, a simultaneous escape problem can be transformed into two separate fixed-ordering escape problems. However, it is very difficult to get the correct ordering. Only a slight difference in the ordering can cause a huge difference in routability. Figure 3.11 shows the escape results of one problem under different net orderings. We use upward mode for both cases. When the ordering is correct, we can get all 6 nets routed. However, if we switch Net 2 with Net 6, we can only route 2 nets.

Owing to the complicated escape situations on dense boards, it is always difficult to find the correct ordering beforehand. Therefore, we use a dynamic strategy to solve this ordering problem. The idea is to gradually determine the routing order as we route the nets. Whenever routing a new net, we tentatively route each remaining net, evaluate the routing cost and pick the one with minimum cost. Here α is the number of pins trapped (unroutable) by routing current net. β is the number of pins blocked (but still routable) by current routing. A net is *blocking* a pin if it is blocking the projection from the pin to the escape boundary. The blockage will cause a twisted route, which usually costs more routing resource. Since trapped pins have more

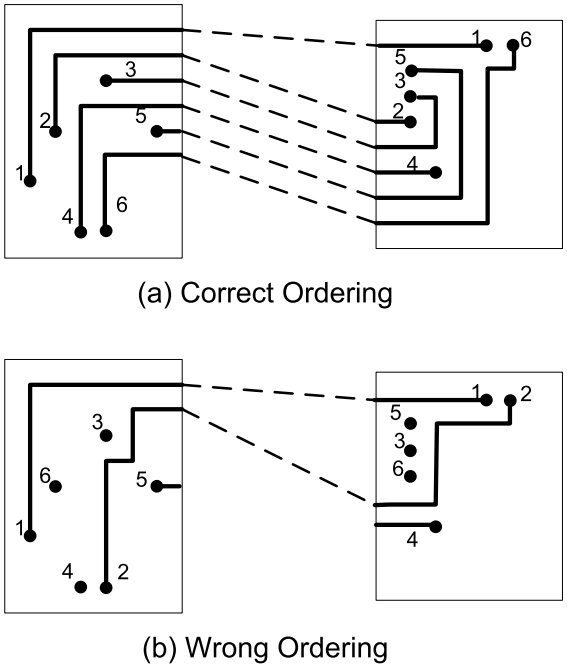


Figure 3.11: Importance of correct ordering.

impact on the routability than blocked pins, α is the dominating factor when comparing two costs. β is compared only when the α values of the two costs are the same. Figure 3.12 is an example illustrating the vector cost function. Suppose we are using upward mode and the first two nets have been routed. Now we tentatively route Net a and calculate its cost. Note that although we are only showing one component to demonstrate the cost idea, each cost element takes both components into account.

The pseudo code of B-Escape is shown in Figure 3.13. B-Escape loops through 6 routing modes. Under each routing mode, we define the process of routing one net (Lines 3-15) as one step. In each step, we first do trial route (Lines 3-8); i.e., tentatively route each remaining net, get the routing cost and then clear the route. Then we pick the net with minimum cost (Line 9) and actually route it (Lines 13-14). Note that cost vectors are compared with each other in lexicographical order, meaning that α is the most significant followed by β. Although most of the time this flow works pretty well, it is possible that we may find one or two nets trapped at a certain step. Line 10 and Line 11 are used to detect the trap and get a new ordering.

We use the following method to reorder. For each step, we keep a list of candidate nets with non-decreasing routing cost. At first, we always choose

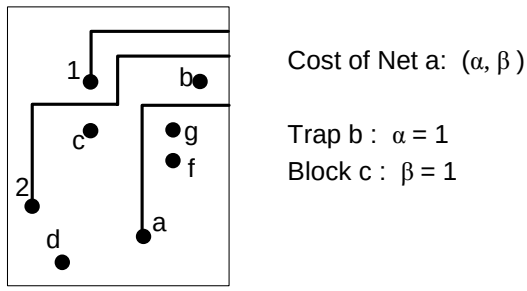


Figure 3.12: Cost function.

```
for each of the six routing modes do
  repeat
    for each unrouted net i do
      route net i in the left component
      route net i in the right component
      calulate the cost vector for net i
      clear the routes generated for net i
    end for
    choose the net j with minimum cost
    if net j traps other nets then
      backtrack and reorder
    else
      route net j in the left component
      route net j in the right component
    end if
  until all routed or exceed the backtrack limit
  store the solution for this routing mode
end for
output the solution with the best routability
```

Figure 3.13: B-Escape routing algorithm.

the net with minimum cost. Once we reach the reordering point, we backtrack to the step where the cost difference between the chosen net and the next candidate net is minimum. Then we choose the next candidate net instead and continue the routing process. If more than one step has the minimum cost difference, we choose the step nearest to the reordering point. Figure 3.14 demonstrates this process. At the first step, the nets are ordered as a, c, b, d, e according to their costs. Thus, we choose Net a as the first net and go to Step 2. This process continues until we reach Step 4, where if we route Net c, some nets will get trapped. Therefore, we need to backtrack and find a new ordering. We calculate the cost difference between the current

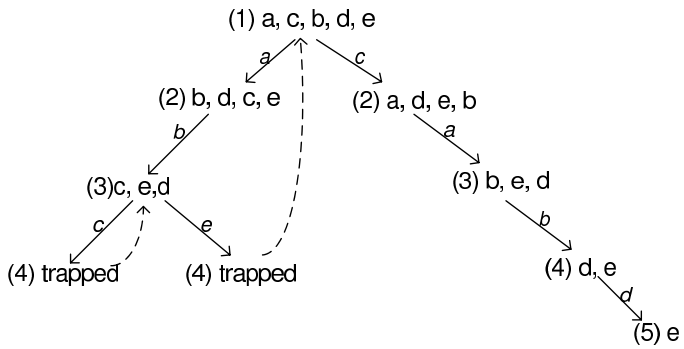


Figure 3.14: Backtrack and reorder.

net and the next candidate net for each step, i.e. $|cost(c) - cost(e)|$ of Step 3, $|cost(b) - cost(d)|$ of Step 2, and $|cost(a) - cost(c)|$ of Step 1. Suppose Step 3 has the minimum cost difference. We go back to step 3 and route Net e instead. Continue this process until either we find the solution as in this figure or the times of backtracking have exceeded the limit. The latter case means that current routing mode cannot solve this particular escape problem; therefore, we switch to the next routing mode.

3.4 Application to PCB Routing

The components on a printed circuit board also present a regular grid structure, as shown in Figure 3.15(a). The labeled pins need to escape to the component boundary. There are usually two routing tracks between each pair of neighboring pins. These tracks present a grid structure similar to the one in Section 3.2. The only difference is that here the pins are not on the routing grid. But we can simply connect a pin to a nearest grid point in the NW, NE, SW or SE direction. The limitation of grid structure is that it cannot represent the diagonal routing on boards, like the one in Figure 3.15(b). Therefore, we add a switch box inside each tile formed by 4 adjacent pins. Figure 3.16(a) shows the grid structure with switch boxes. Each switch box has 12 points. Each corner point corresponds to a neighboring pin; e.g., Point 1 corresponds to Pin a, Point 7 corresponds to Pin c and Point 10 corresponds to Pin b. The other points correspond to track segments. Figure 3.16(b) shows all the possible connections between Point 1 and the other points on the switch box. In the routing process, whenever

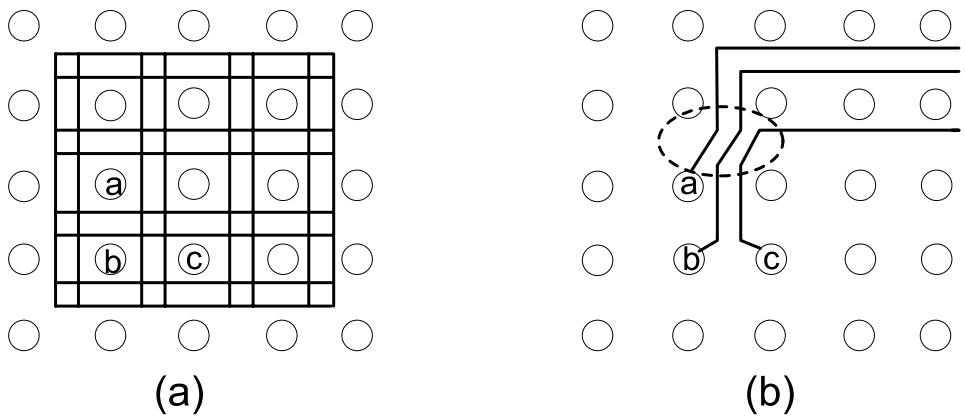


Figure 3.15: Grid structure on a PCB component. This structure cannot be used to represent the diagonal routing in (b).

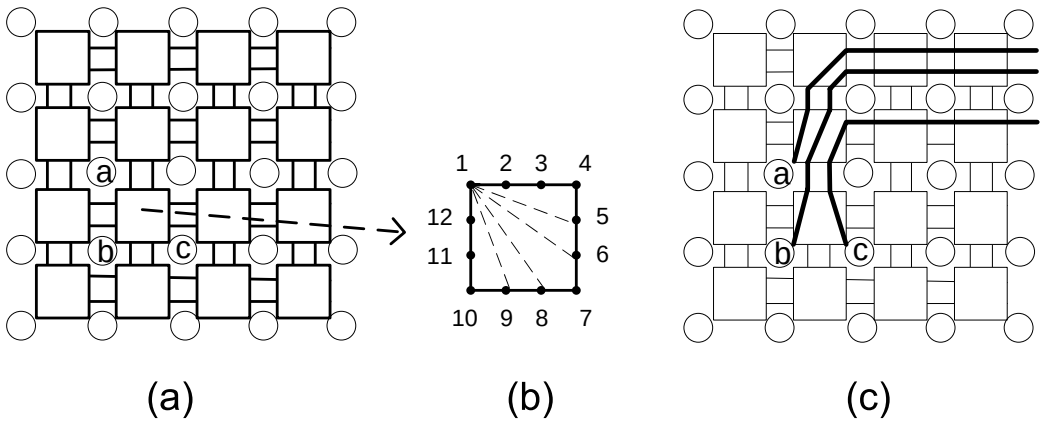


Figure 3.16: Grid structure with switch boxes.

a new connection needs to be made within a switch box, we check whether this connection is crossing with previous connections, and if not, whether the spacing rule is satisfied. For example, if Point 1 and Point 8 are already connected by previous routes, the connection between Point 5 and Point 12 is disabled; otherwise, it will cause crossing within the switch box. Figure 3.16(c) is the routing solution obtained by using switch boxes.

B-Escape can easily handle differential pairs. A *differential pair* is a pair of nets that are required to be routed together. The routing of a differential pair is called *pair routing*. Figure 3.17 shows three routing solutions, where Net $a1$ and Net $a2$ are a differential pair. (Note that for conciseness of illustration, we omit the underlying routing grid and switch boxes in the figure.) The first one satisfies the pair constraint. The second one is illegal because the

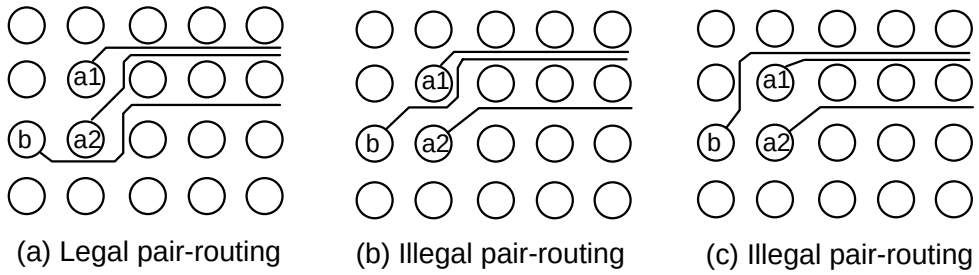


Figure 3.17: Pair-routing solutions.

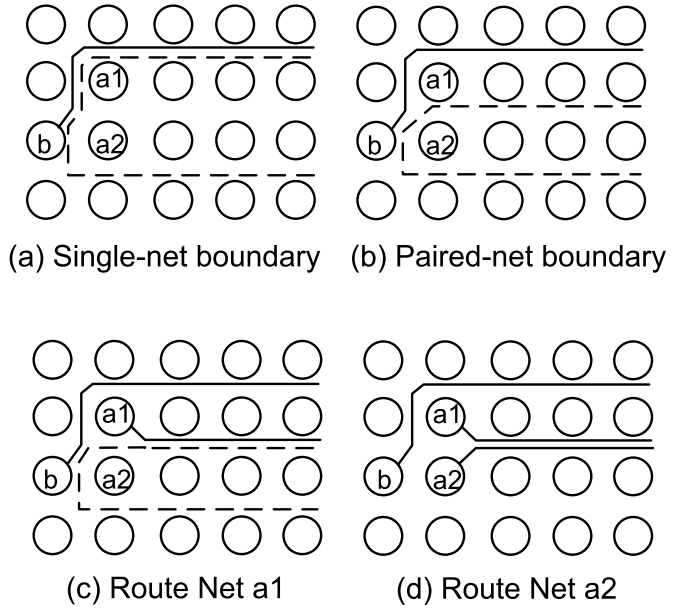


Figure 3.18: Routing boundary for pair-routing.

paired nets are separated by another net. The third one is also illegal because the paired nets are separated by a row of pins. In B-Escape, as long as the paired nets are consecutively indexed, they are always routed side by side because the second net is basically following the boundary defined by the first net. To avoid the situation in Figure 3.17(c), we maintain two routing boundaries. For non-paired nets or single nets, we use the boundary exactly defined by the previously routed nets as shown in Figure 3.18(a). Figure 3.18(b) shows how to adjust the single-net boundary for pair-routing. Here the upper boundary is lowered so that there is one track right beneath the boundary. Figure 3.18(c) shows the routing of Net $a1$ obtained by tracing the paired-net boundary. Once $a1$ is routed, Net $a2$ can be treated as a single net and its route is finished in Figure 3.18(d).

3.5 Solve Large Problems by Clustering

Our router performs very well for small and medium sized problems. For large routing problems, our router would require more time to find a solution because the dynamic reordering mechanism (Section 3.3) takes longer to figure out a proper routing order. On the other hand, pins in large routing problems usually present a clustering structure due to the internal bus structure of the problem. Pins belonging to the same cluster are located close to each other. By taking advantage of such clustering structure, we can significantly reduce the runtime for large problems. Our main idea is as follows (see also Figure 3.19): First, we use a clustering algorithm to discover the pin clusters. Then we plan the global routes for each cluster. Finally, we detail route each cluster by the B-Escape router following the global routing.

3.5.1 Clustering

Our clustering algorithm is based on the idea of K-closest neighbor graph. A *K-closest neighbor graph* K-CNG (V, E) is a directed graph with V being the vertex set and E being the edge set. Each vertex corresponds to a pin in one component. An edge (p, q) is included in the graph if and only if there are less than K vertices in V which are closer to p than q. In this case, q is called p's *K-closest neighbor*. Figure 3.20 illustrates an example of 2-closest

neighbor. In the figure, Pins 2, 3 and 4 have the smallest distance to Pin 1. Therefore, they are all Pin 1's 2-closest neighbors. Pin 5 is not a 2-closest neighbor of Pin 1 because there are already three pins closer than it to Pin 1. As a result, we would have edges from 1 to 2, 3 and 4 respectively, but no edge from 1 to 5. After we build the K-CNG (V, E_L) and K-CNG (V, E_R) for the left and right components respectively, we compute the intersection graph K-CNG (V, $E_L \cap E_R$) (see Figure 3.21). If two pins are connected in the intersection graph, it means that they are relatively close to each other in both components and they should belong to the same cluster. We consider each connected component in the intersection graph as a cluster. For example, there are three clusters in Figure 3.21(c).

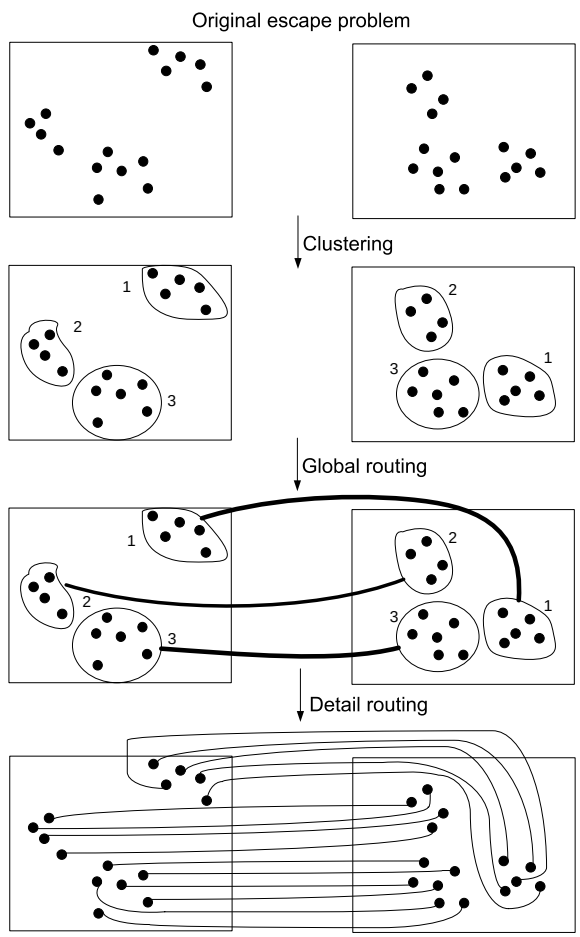


Figure 3.19: Routing a large problem by clustering.

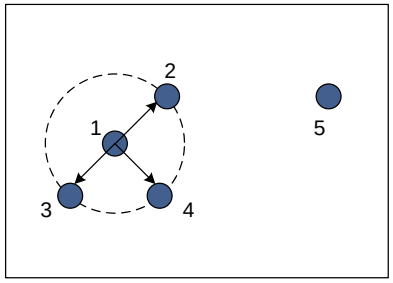


Figure 3.20: Building 2-CNG (V,E) on one component. Only the edges from Pin 1 are shown.

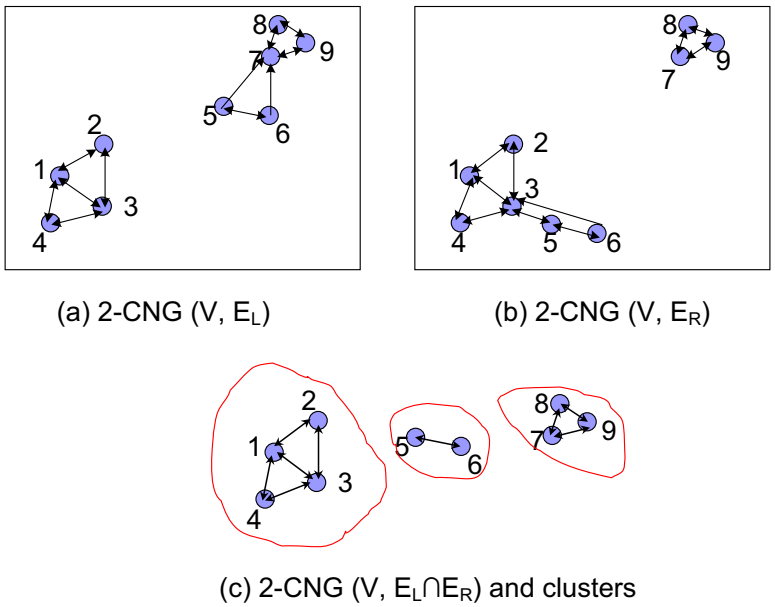


Figure 3.21: Building an intersection graph. Each connected component in the graph corresponds to one cluster.

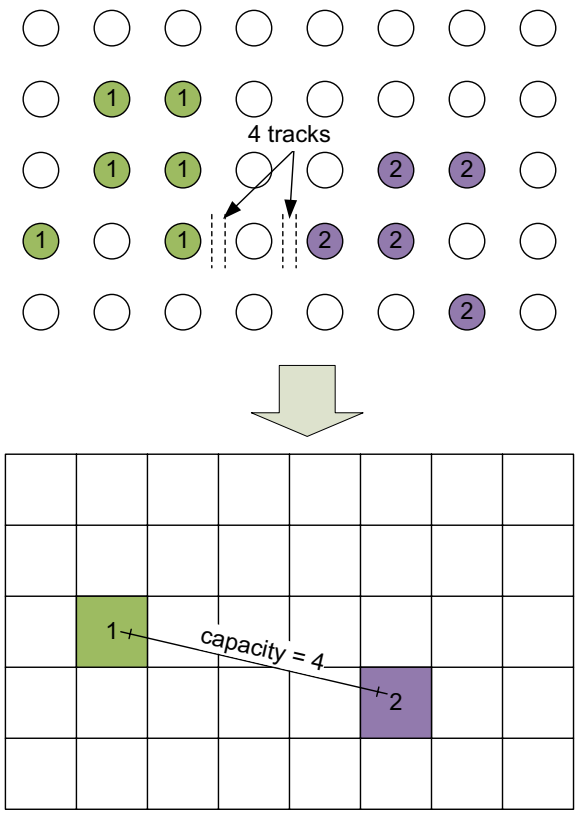


Figure 3.22: Contracting clusters into representative pins and adding capacity constraints between the representative pins. We assume two routing tracks between adjacent pins.

3.5.2 Global Routing

After the clusters are discovered by the previous step, we need to find out a global routing of the clusters to guide the detail routing. We abstract the global routing problem into a regular pin-to-pin routing problem. First, we contract each cluster into a representative pin which is located at the centroid of the cluster (that is, by averaging the coordinates of all the pins in the cluster). Then we define the wiring capacity as the number of wiring tracks between the closest points of the two clusters. This wiring track number limits the number of wires that can pass between the two clusters and thus becomes a capacity constraint. Figure 3.22 shows how we contract pins and compute the capacity.

Now we have a pin-to-pin routing problem: connect the corresponding representative pins while satisfying the capacity constraint. We solve this routing problem by a negotiated congestion based router [36].

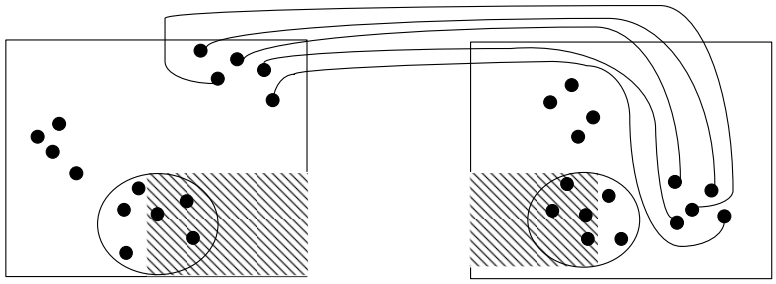


Figure 3.23: Projection of cluster 3 onto its escape boundaries.

3.5.3 Detail Routing

Once we have a global routing for the clusters, we will perform detail routing for each cluster following the global routing. The global routing gives us two essential pieces of information: the escape ordering of the clusters around the component boundary and which boundary each cluster escapes toward. In our detail routing phase, the order in which the clusters are routed is consistent with the order of the global routing around the boundary. In Figure 3.19, the order of the global routing around the boundary would be 1 followed by 2 and then 3.

When we route one cluster, other clusters that are already routed are considered as blockages. In addition, we also need to reserve areas that might be used by the unrouted clusters in the future. To estimate the possible routing area of an unrouted cluster, we project the unrouted cluster to the component boundary it escapes toward in the global routing. Such a projection creates a rectangular area that will be considered as blockage for the current cluster. For example, the shaded rectangle in Figure 3.23 represents the projection of Cluster 3 of Figure 3.19, which is blocked when we are routing Cluster 2.

The clustering strategy presented in this section is very effective for solving large escape routing problems. As will be shown in the experimental results, the use of this strategy greatly reduces the runtime of our escape router.

3.6 Experimental Results

We have implemented B-Escape in C++ and carried out the experiments on a Pentium 4 2.8 GHz system with 4 GB memory. We tested B-Escape on 14 industrial benchmark escape problems. These problems were previously

successfully routed manually by experienced layout engineers taking about 8 hours per problem. Table 3.1 gives detailed information on the benchmark problems as well as the comparison results with Cadence Allegro PCB router. (#Row and #Col are the number of rows and columns of the routing grid.) B-Escape successfully solved all 14 problems but Cadence Allegro router only completed the routing of half of them. (Note that if an auto router fails to route even just one single net, the manual effort needed to finish the routing could be as high as re-routing the entire problem from scratch.) B-Escape is also very fast. Unfortunately we could not directly compare the runtime with Allegro since there is no way to separate the escape routing runtime from the total execution time, which includes runtime for the detailed routing of inter-component connections. Since industrial PCB routers are typically based on the framework of cost-driven negotiated-congestion routing (NCR), we implemented an escape router based on NCR for runtime comparison. Our NCR router is comparable to Allegro in performance, solving 7 out of the 14 problems. On average, B-Escape is 15 times faster than NCR. (See Table 3.2.) Figure 3.24 shows the output routing solution of a simultaneous escape routing problem by B-Escape. (Note that this problem is not among the 14 industrial problems we used in the experiments since we do not have the permission to display any of them. Instead we display a problem which is derived from an industrial problem.) Due to the nature of the various detouring modes of B-Escape, the original routing solution contains many long detouring wires. We implemented a compaction procedure to shorten the long wires. The compaction simply searches for the shortest paths while maintaining the topology achieved by B-Escape and it is usually done in seconds. The solution displayed in Figure 3.24 is obtained after compaction. Note that even after compaction, B-Escape routing may still generate longer routes than the shortest-path based routing scheme. However, compared with inter-component connection, escape routing only contributes to a small portion of the total wire-length. Moreover, it is typical that there are min-max length constraints requiring wires to meander to increase its wire-length. Longer escape routes can actually be beneficial since they reduce the number of length extensions in the inter-component connections.

We also implemented the clustering strategy presented in Section 3.5. We ran our router on 3 large escape routing problems with and without using the clustering strategy. Both approaches achieved 100% routability on all 3

problems. However, we observed a substantial speedup when the clustering strategy was used. Table 3.3 shows the benchmarks we used to test the clustering strategy. Table 3.4 summarizes the experimental results. We can see that the speedup can be as great as 40x. The reason why the clustering strategy can greatly cut down the runtime is that by decomposing the routing problem into smaller clusters, the solution space for proper routing order is greatly limited. As a result, the backtracking based dynamic ordering step takes much less time to find a good routing order. We also tested the negotiated-congestion router on the three large escape problems. The routability is equal to or less than 55% and the runtime ranges from 6 hours to 11 hours.

Note that although all of our test cases have two routing tracks between adjacent pins, our algorithm is not limited to the particular number of tracks. More/fewer tracks only result in larger/smaller routing grids. And our switch-box structure provides the flexibility to handle any spacing rule

Table 3.1: Benchmark property and comparison results with Allegro

		left component	right component	routability	
data	#nets	#row×#col	#row×#col	Allegro	B-Escape
Ex1	39	44×20	22×28	100%	100%
Ex2	36	30×18	42×16	100%	100%
Ex3	18	18×24	18×16	100%	100%
Ex4	26	28×32	28×26	95%	100%
Ex5	52	24×26	60×10	80%	100%
Ex6	40	16×14	22×16	100%	100%
Ex7	64	38×16	36×12	90%	100%
Ex8	32	34×12	34×12	100%	100%
Ex9	41	24×14	30×30	100%	100%
Ex10	37	18×28	24×10	95%	100%
Ex11	58	54×12	62×12	96%	100%
Ex12	36	34×28	20×26	100%	100%
Ex13	38	26×28	36×30	70%	100%
Ex14	39	38×36	32×26	80%	100%
# of routed problems				7/14	14/14

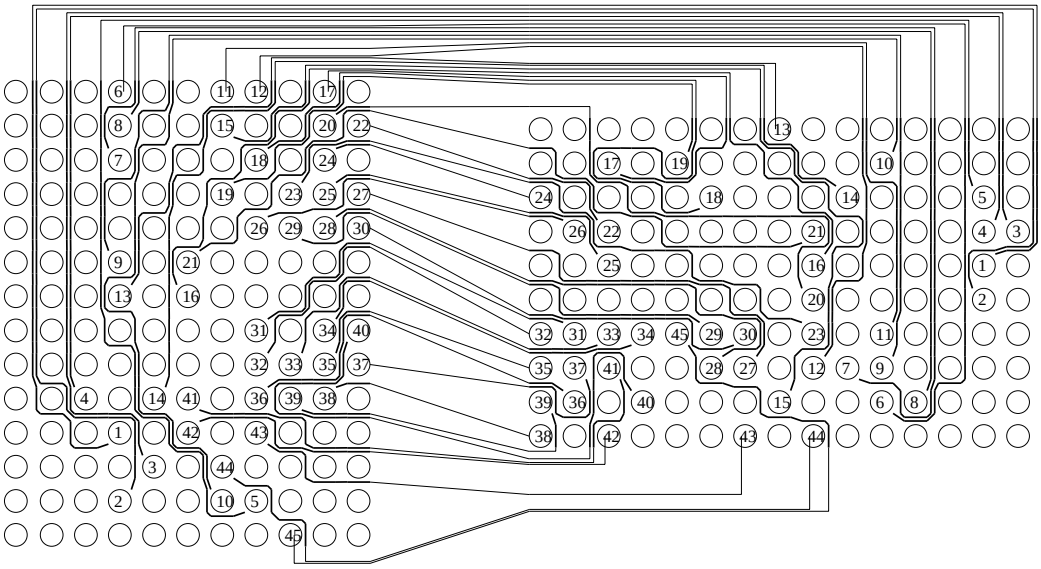


Figure 3.24: Escape solution by B-Escape.

Table 3.2: Running times(sec) of B-Escape and NRC

	Ex1	Ex2	Ex3	Ex4	Ex5	Ex6	Ex7
B-Escape	106.8	87.4	0.2	81.0	185.1	82.3	691.3
NRC	2872.0	520.5	55.8	403.5	5442.0	940.2	2881.0

	Ex8	Ex9	Ex10	Ex11	Ex12	Ex13	Ex14
B-Escape	0.8	8.7	96.2	44.0	74.1	46.5	289.0
NRC	18.5	3610.0	6189.0	629.8	2827.0	1005.0	2580.0

Table 3.3: Benchmarks for testing the clustering strategy

data	#rows	#cols	#rows	#cols	#clusters	#nets
Large1	35	35	35	35	4	94
Large2	30	20	35	11	4	93
Large3	40	40	40	40	3	71

Table 3.4: Speedup of the clustering strategy over the flat run

	runtime		
data	with clustering (s)	w/o clustering (s)	speedup
Large1	116	1648	X14.2
Large2	155	1228	X10.6
Large3	106	4740	X40.9

for the 45 degree routing segments.

3.7 Conclusion

We have presented a new simultaneous escape routing algorithm in this chapter. It dynamically decides net ordering according to real time congestion information. Each net is routed by a novel boundary routing engine, which always leaves as much space as possible to the unrouted nets and therefore guarantees great routability. Experimental results show that our escape router performs better than the Cadence PCB router on industrial benchmarks. We also proposed a clustering strategy for solving large escape routing problems. The strategy can discover the clustering structure in large problems, and by utilizing such structure, it can greatly reduce the runtime. The effectiveness of our clustering strategy is verified by experiments.

CHAPTER 4

GPU IMPLEMENTATION OF BFS

4.1 Introduction

The graphics processing unit (GPU) has become a popular cost-effective parallel platform in recent years. Although parallel execution on a GPU can easily achieve speedup of tens or hundreds of times over straightforward CPU implementations, to accelerate intelligently designed and well optimized CPU algorithms is a very difficult job. Therefore, when researchers report exciting speedup on GPUs, they should be very careful about the baseline CPU algorithms. Fig. 4.1 illustrates the performance difference between the 1000x accelerated n^2 algorithm and the $n\log n$ algorithm. After n grows to a certain point, the greatly accelerated n^2 algorithm becomes slower than the un-accelerated $n\log n$ algorithm. Hence the speedup is only meaningful when it is over the best CPU implementation. A similar observation was also made in [37]. Breadth-first search (BFS) is a graph algorithm that has extensive applications in computer-aided design as well as in other fields. However, it is well known that accelerating graph algorithms on GPUs is very challenging and we are aware of only two published works [38], [39] on accelerating BFS. Harish and Narayanan pioneered the acceleration of BFS on the GPU [38]. This work has great impact in high performance computing and graph algorithm areas. It has been cited not only by research papers but also in NVIDIA CUDA Zone and university course materials. However, for certain types of graphs, such as sparse graphs with many BFS levels, this work is still slower than the fastest CPU program. The other work [39] showed 12 to 13 times speedup over a matrix-based BFS implementation, but it should be noted that such BFS implementation is slower than the traditional BFS algorithm of [40].

This chapter will present a new GPU implementation of BFS. It uses a

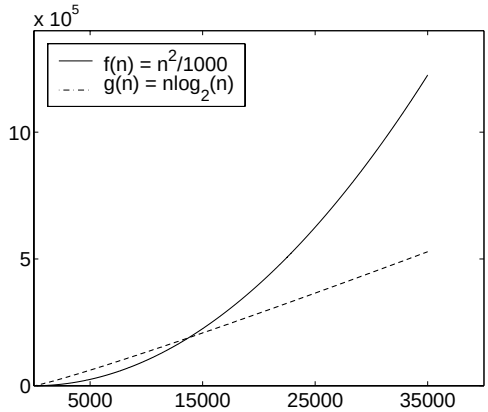


Figure 4.1: Performance comparison between 1000x accelerated n^2 algorithm and $n\log n$ algorithm.

hierarchical technique to efficiently implement a queue structure on the GPU. A hierarchical kernel arrangement is also proposed to reduce synchronization overhead.

4.2 BFS and Previous Works

Given a graph $G = (V, E)$ and a distinguished *source* vertex s, breadth-first search (BFS) systematically explores the edges of G to discover every vertex that is reachable from s. We use $V(E)$ to represent both the vertex (edge) set and the number of vertices (edges). BFS produces a breadth-first tree with root s that contains all reachable vertices. The vertices in each *level* of the tree compose a *frontier*. *Frontier propagation* checks every neighbor of a frontier vertex to see whether it is visited already; if not, add it into the new frontier. Fig. 4.2 shows an example operation of BFS. The traditional BFS algorithm outlined in [40] uses a queue structure to store the frontier. Its computational complexity is $O(V + E)$. For sparse graphs with $E = O(V)$, the complexity of BFS is $O(V)$.

Harish and Narayanan presented the first work of implementing BFS on GPU [38]. (Hereafter, we will refer to this work as IIIT-BFS.) It points out that to maintain the frontier queue can cause a huge overhead on GPU. Hence they eliminate the frontier structure. Instead, for each level, they exhaustively check every vertex to see whether it belongs to the current frontier. Let L be the total number of levels; then the total time spent

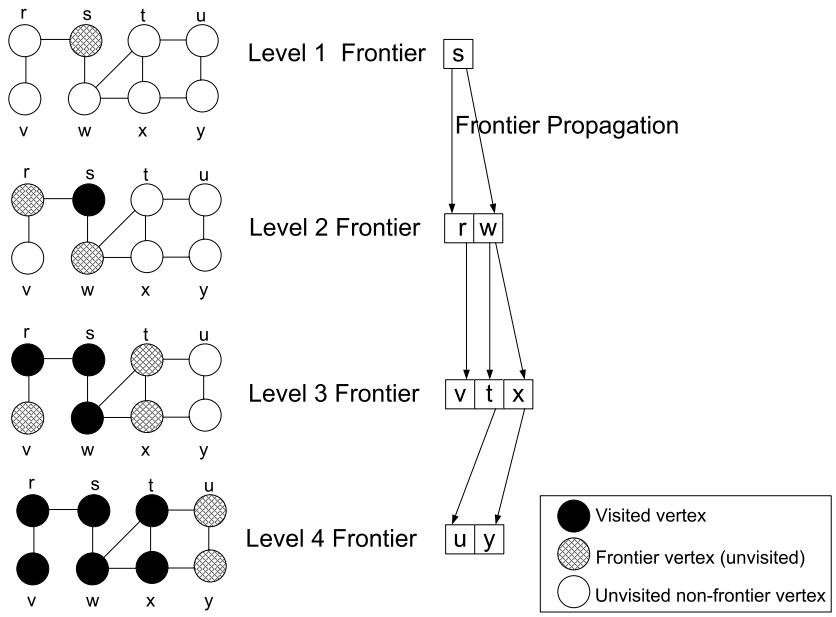


Figure 4.2: The operation of BFS on an undirected graph.

to check frontier vertices is $O(VL)$. The time to explore every edge of the graph is $O(E)$. Therefore, the time complexity of IIIT-BFS is $O(VL + E)$, higher than $O(V + E)$ of the CPU algorithm. In the worst case scenario, we have $L = O(V)$. It means a complexity of $O(V^2 + V) = O(V^2)$ for sparse graphs, much slower than the $O(V)$ implementation. Based on our analysis in Section 4.1, the accelerated version of a slow algorithm is still worse than a fast sequential algorithm. Our analysis will be further verified by the experimental results shown in Section 4.5. Of course, when the graph is dense with $E = O(V^2)$, or when L is very small, IIIT-BFS can be faster than the sequential program.

The second work [39] accelerated a BFS algorithm implemented by matrix-vector multiplication. Let A be the adjacency matrix of a graph; i.e., an entry (i, j) is equal to 1 if vertex i has an edge toward vertex j; 0 otherwise. Let vector x record the status of each vertex. $x(i)$ is larger than 0 if vertex i has been reached in the latest level of propagation; 0 otherwise. Initially only the entry of the source vertex in x is non-zero. Then each frontier propagation corresponds to a matrix-vector multiplication ($x \leftarrow Ax$). Fig. 4.3 shows the first level of propagation on the graph of Fig. 4.2. The running time of each matrix-vector multiplication is $O(E)$. In addition, it takes $O(V)$ time to initialize the vector. Therefore the total running time is $O(V + EL)$, higher

$$
A^T x = \begin{array}{c} \\ r \\ s \\ t \\ u \\ v \\ w \\ x \\ y \end{array}
\begin{array}{c}
\begin{array}{cccccccc} r & s & t & u & v & w & x & y \end{array} \\
\begin{bmatrix}
0 & 1 & 0 & 0 & 1 & 0 & 0 & 0 \\
1 & 0 & 0 & 0 & 0 & 1 & 0 & 0 \\
0 & 0 & 0 & 1 & 0 & 1 & 1 & 0 \\
0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\
1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \\
0 & 1 & 1 & 0 & 0 & 0 & 1 & 0 \\
0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 \\
0 & 0 & 0 & 1 & 0 & 0 & 1 & 0
\end{bmatrix}
\end{array}
\begin{bmatrix} 0 \\ 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}
=
\begin{array}{c} \begin{bmatrix} 1 \\ 0 \\ 0 \\ 0 \\ 0 \\ 1 \\ 0 \\ 0 \end{bmatrix} \end{array}
\begin{array}{c} \\ r \\ s \\ t \\ u \\ v \\ w \\ x \\ y \end{array}
$$

Figure 4.3: Frontier propagation by matrix-vector multiplication. This corresponds to the first frontier propagation on the graph from Fig. 4.2.

than $O(V+E)$. Again in the worst case, we have $L = O(V)$ and the running time is $O(V + EL) = O(V^2)$. Hence the same argument about IIIT-BFS applies here. For large graphs, the accelerated matrix-based BFS will still be slower than the traditional sequential BFS.

Lauterbach et al. solved a different BFS problem in [41]. Under the framework of BFS, they do intensive computation on each node. This work uses a compaction technique to maintain the queue structure on the GPU. However, the compaction will add intolerable overhead to our BFS problem.

There are also parallel BFS algorithms proposed for other parallel architectures such as [42], [43], [44]. However, since different architectures have different efficiencies for synchronization, atomic operations, memory accesses, etc., the parallelism ideas applicable to those architectures may not be used on the GPU. As far as we know, the ideas proposed in this chapter were never used on other architectures.

4.3 GPU Architecture

The NVIDIA GT200 graphics processor is a collection of 30 multiprocessors, with 8 streaming processors each (Fig.4.4). We use NVIDIA CUDA computing model to do parallel programming on GPU. A typical CUDA program consists of several phases that are either executing on host CPU or on GPU. The CPU code does the sequential part of the program including file I/O, allocating and releasing global memory, memory copy between CPU and GPU, etc. The CPU code is straight ANSI C code. The phases that exhibit a rich

amount of parallelism are usually implemented in GPU code, called *kernels*. The action of calling a GPU function from CPU code is termed as *kernel launch*. When a kernel is launched, a two-level thread structure is generated. The top level is called *grid*, which consists of one or more *blocks*. Each block consists of the same number (at most 512) of threads. The whole block will be allocated to one multiprocessor.

In order to better explain our GPU solution, we also need to introduce the thread scheduling scheme on GT200. Once a block is assigned to a multiprocessor, it is further divided into 32-thread units, called *warps*. The unit of thread scheduling is a warp. Whenever a warp is scheduled, its first 8 threads will be executing on 8 streaming processors in parallel. After that the following 8 threads will be executing, and so on. Hence it takes 4 clock cycles to finish one instruction in the 32 threads of each warp.

CUDA threads may access data from multiple memory spaces during their execution. Each thread has private local memory. Each thread block has shared memory visible to all threads of the block and with the same lifetime as the block. Note that the shared memory is on-chip memory. To access a shared memory only takes 2 clock cycles compared with approximately 300 clock cycles for off-chip device memory. The device memory is accessible to all the threads of a grid. There are three types of memory in the device memory space: global memory, constant memory and texture memory. Global memory is not cached, so it is very important to follow the right access pattern to achieve maximum memory bandwidth. Memory bandwidth is used most efficiently when consecutive threads are accessing consecutive locations in global memory. In this case, memory transactions of threads from half-warp can be *coalesced* into one transaction. This is called *memory coalescing*. Constant memory and texture memory are both read-only memory and they both have cache, although much smaller than the CPU cache. The constant memory read also follows the above mentioned coalescing rule, but the constant memory is much smaller than the global memory. Texture memory is mostly used when the memory reads do not follow the access patterns that global or constant memory reads must respect to get good performance.

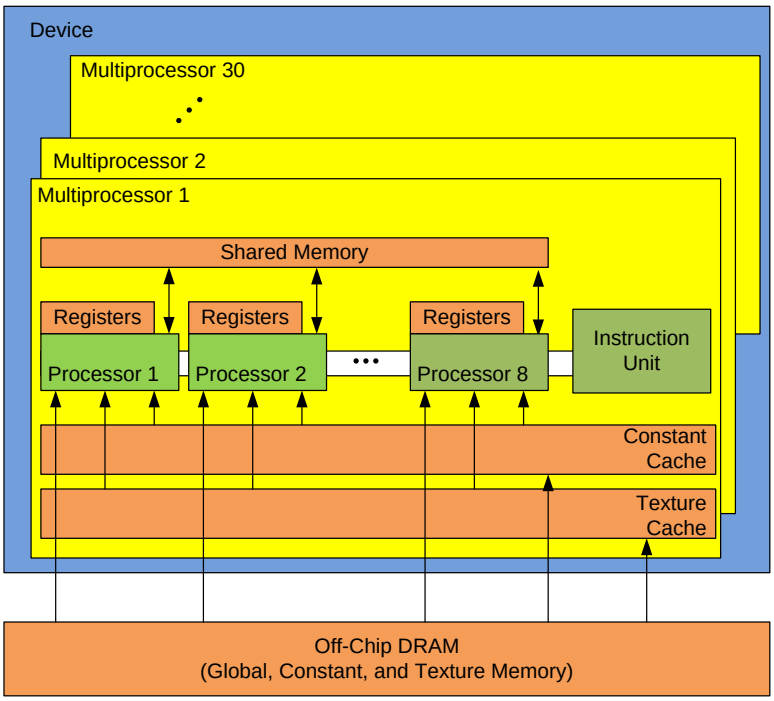


Figure 4.4: Basic organization of the NVIDIA GT200 GPU.

4.4 Our GPU Solution

The BFS process propagates through the graph by levels. In each level, there are two types of parallelism: one is to propagate from all the frontier vertices in parallel, and the other is to search every neighbor of a frontier vertex in parallel. Since a lot of EDA problems such as circuit simulation, static timing analysis and routing are formulated on sparse graphs, we do not expect many neighbors for each frontier vertex. Therefore, our implementation of BFS only explores the first type of parallelism; i.e., each thread is dedicated to one frontier vertex of the current level.

Algorithm 1 is the pseudo code of our GPU implementation, called UIUC-BFS. Here *cur-f* represents the frontier in current level and *new-f* represents the new frontier in the next level. Note that Algorithm 1 runs on CPU. It calls the kernel function Algorithm 2 which runs on the GPU. Section 4.4.1 will give more details about the implementation of Line 6 in Algorithm 2. Section 4.4.2 presents techniques to further reduce the overhead of GPU-BFS. Therefore, the final version of UIUC-BFS will be slightly different from Algorithm 1.

Algorithm 1 UIUC-BFS (G, s)

```
1: cur-f = {s}
2: while cur-f ≠ φ do
3:    new-f = φ
4:    Launch Propagation-Kernel (G, cur-f, new-f)
      {each vertex of cur-f corresponds to one thread}
5:    cur-f = new-f
6: end while
```

Algorithm 2 Propagation-Kernel (G, *cur-f*, *new-f*)

```
1: tid = id of the current thread in the grid
2: u = cur-f[tid]
3: Visit u
4: for every neighbor v of u do
5:    if v is not visited then
6:       Add v into new-f
7:    end if
8: end for
```

4.4.1 Hierarchical Queue Management

It is difficult to maintain the new frontier in a queue because different threads are all writing to the end of the same queue and end up executing in sequence. Fig. 4.5 illustrates this situation. In the remaining part of this section, we call this a *straightforward implementation*. To avoid the collision on the queue, we introduce a hierarchical queue structure. The idea is that once we have quickly created the lower-level queues, we will know the exact location of each element in the higher-level queue, and therefore copy the elements to the higher-level queue in parallel.

A natural way to build the frontier hierarchy is to follow the two-level thread hierarchy; i.e., build the grid-level frontier based on block-level frontiers. A block-level frontier can be stored in the fast shared memory. However, this strategy still cannot avoid the collision at the block level. Hence we add another level – warp level – into the hierarchy and completely eliminate collisions on the warp-level queues. Fig. 4.6 shows the overall hierarchy.

A *W-Frontier* is defined as a frontier only accessed by certain threads from a warp. A warp is the scheduling unit in the GPU, which is composed of 32 consecutive threads. Since there are only 8 streaming processors within each multiprocessor, a warp is further divided into four 8-thread groups. Fig. 4.7

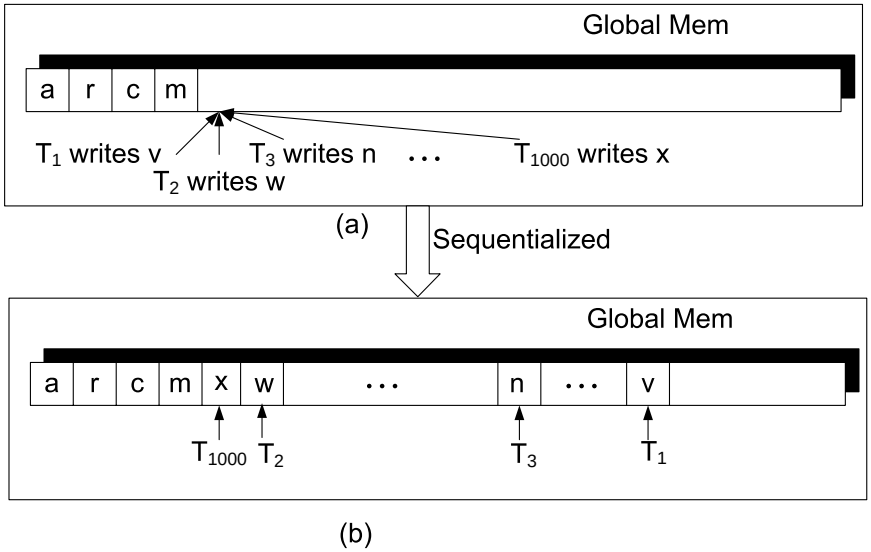


Figure 4.5: Straightforward implementation of the frontier queue. (a) Conflicts happen when multiple threads are writing to the end of the queue. (b) Conflict resolution result: threads are writing to arbitrary locations in the queue.

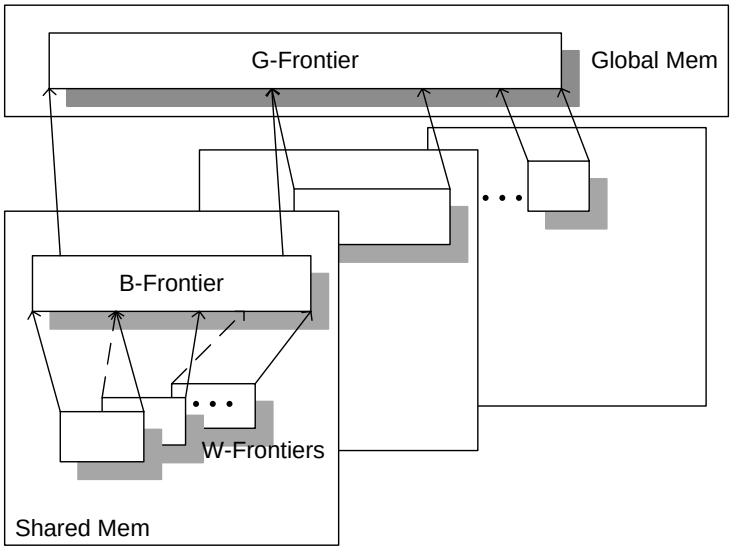


Figure 4.6: Hierarchical frontiers.

illustrates how threads are scheduled along the timeline. The vertical lines represent the clock boundaries. W_iT_α represents Thread α from Warp i. We do not know the scheduling order among warps, but once a warp is scheduled, it always runs T_1-T_8 first, followed by T_9-T_{16}, T_{17}-T_{24}, and finally T_{25}-T_{32}. In GT200, threads in the same row of Fig. 4.7 never execute simultaneously. If each row of threads write to one W-Frontier queue, we can guarantee no collision. This scheme is portable as long as the warp size assumption is parameterized. Note that writing to a queue actually includes two steps; i.e., write an element to the end of the queue and update the end of the queue. To guarantee the correctness in parallel execution, we use an atomic operation, so that we can get the current end location and update it in one instruction.

A *B-Frontier* is defined as the frontier common to a whole block. It is simply the union of 8 W-Frontiers. To copy the W-Frontier elements into B-Frontier in parallel, we need indices of each element in the B-Frontier. Indeed, we only need 8 offset values for this purpose, i.e., the offset values of each W-Frontier in the B-Frontier. Algorithm 3 calculates these offset values. Here |W-Frontiers[i]| is the number of elements (vertices) in the frontier. With only 8 W-Frontiers, we use one thread to calculate the offsets for all W-Frontiers in the B-Frontier. Then the index of an element in the B-Frontier is simply its index within its W-Frontier plus the offset of its W-Frontier. Note that both the W-Frontiers and the B-Frontiers reside in fast shared memory.

Algorithm 3 Calculate-Offsets (W-Frontiers, *offsets*)

```
1: if the thread id is 0 then
2:     offsets[0] = 0
3:     for i from 1 to 7 do
4:         offsets[i] = offsets[i − 1] + |W-Frontiers[i − 1]|
5:     end for
6: end if
```

Finally, a *G-Frontier* is defined as the frontier shared by all the threads of a grid. In other words, the G-Frontier stores the complete new frontier. A G-Frontier resides in global memory and consists of B-Frontiers from all the blocks. We use atomic operations to obtain the offsets of B-Frontiers so that parallel copying from B-Frontiers to the G-Frontier can be performed. Note that in the parallel copy, it is natural to have consecutive threads writing to consecutive locations in the G-Frontier; thus, memory coalescing is guaran-

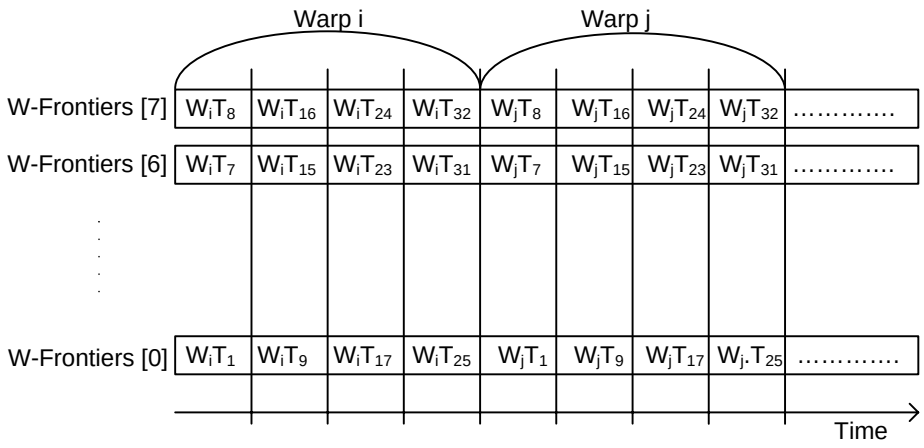


Figure 4.7: Thread scheduling and W-Frontier design.

teed. For the sake of comparison, recall the straightforward implementation in Fig. 4.5. Threads are accessing arbitrary queue locations, depending on the conflict resolution result; therefore, the straightforward implementation of the queue structure barely has memory coalescing.

4.4.2 GPU Synchronization and Kernel Arrangement

The correct BFS implementation requires thread synchronization at the end of each level. Unfortunately CUDA does not provide global barrier synchronization function across blocks. A general solution is launching one kernel for each level and a global barrier exists between two launched kernels. However, frequent kernel launches can cause a huge overhead. This section presents hierarchical kernel arrangement, where only the highest layer uses this expensive synchronization method and the other ones use efficient GPU synchronization.

GPU synchronization is a synchronization technique without terminating a kernel. It includes intra-block synchronization and inter-block synchronization. In the following discussions, we assume the largest possible number of threads, i.e. 512 threads, in each block.

Intra-block synchronization uses CUDA barrier function to synchronize threads within one block. Hence it only applies to the levels with frontiers no larger than 512 vertices. Note that once a kernel is launched, the number of threads in this kernel cannot be changed anymore. Therefore, when launching

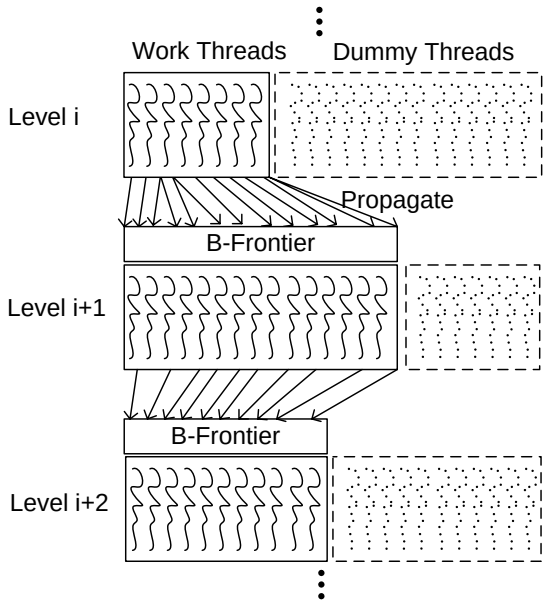


Figure 4.8: The single-block kernel propagates through multiple levels using intra-thread synchronization.

such single-block kernel from the host CPU, we always generate 512 threads regardless of the size of the current frontier. Of course, only the threads with an index smaller than the size of the frontier are propagating; the other threads are called *dummy threads* and do nothing. This kernel can propagate through multiple levels (Fig. 4.8) with an intra-block synchronization at the end of each level. Now that there is only one working block, no G-Frontier is needed any more and global memory access is also reduced.

The second GPU synchronization strategy has a wider application and applies to the levels with frontiers as large as 15360 vertices. *Inter-block synchronization* synchronizes threads across blocks by communicating through device memory. Xiao and Feng introduced the implementation details of inter-block synchronization [45]. To apply this synchronization strategy, the number of blocks should not be larger than the number of multiprocessors. On the GT200 GPU, this means at most $512 \times 30 = 15360$ threads in the grid.

In summary, our GPU implementation has a three-layer kernel hierarchy. The top kernel is launched when the frontier has more than 15360 vertices. It depends on kernel termination and re-launch for synchronization. With more than 15360 vertices, the overhead of launching kernel for each level of propagation becomes acceptable compared to the work performed. The

middle kernel is used for the frontiers with 513 to 15360 vertices, and the bottom kernel is for the frontiers with at most 512 vertices. The last two kernels use GPU synchronization to propagate through multiple levels; therefore will greatly reduce the kernel-launch overhead.

4.5 Experimental Results

In this section, we will compare our GPU-based BFS solution with the fastest CPU-BFS solution [40] as well as the IIIT-BFS solution. We will present the experimental results on two generations of CPU and GPU combinations respectively.

4.5.1 Experiments on GT200 GPU

All experiments were conducted on an NCSA AC cluster node [46] with a 2.4 GHz Opteron processor and 1 MB cache. A single NVIDIA Tesla T10 graphics card with 4 GB global memory was used to run CUDA applications. We first tested the programs on degree-6 "regular" graphs. (For simplicity, we used grid-based graphs. Therefore, only the vertices inside the grids are of degree 6. The very few vertices on the boundaries of the grids have degrees less than 6. And the source vertices are always at the centers of the grids.) The experimental results are shown in Table 4.1. Here UIUC-BFS represents our GPU implementation of BFS, and CPU-BFS is the BFS implementation outlined in [40]. The last column shows the speedup of UIUC-BFS over CPU-BFS. We can see that IIIT-BFS (the source code was obtained from the authors) is slower than CPU-BFS just as we discussed in Section 4.2. On the other hand, UIUC-BFS is faster than CPU-BFS on all the benchmarks. On the largest one, up to 10 times speedup was achieved. We also downloaded the same four graphs as [38] from the DIMACS challenge site [47]. All these graphs have average vertex degree of 2, and the maximum degree can go up to 8 or 9. Again IIIT-BFS is slower than CPU-BFS, and UIUC-BFS achieves speedup over CPU-BFS (Table 4.2). Finally, we tested the BFS programs on scale-free graphs. Each graph was built in the following way. We made 0.1% of the vertices have degrees equal to 1000. The remaining vertices have the average degree of 6 and the maximum degree of 7. Table

Table 4.1: BFS results on regular graphs

#Verte	IIIT-BFS (ms)	CPU-BFS (ms)	UIUC-BFS (ms)	Sp.
1M	462.8	146.7	67.8	2.2
2M	1129.2	311.8	121.0	2.6
5M	4092.2	1402.2	266.0	5.3
7M	6597.5	2831.4	509.5	5.6
9M	9170.1	4388.3	449.3	9.8
10M	11019.8	5023.0	488.0	10.3

4.3 shows the running times of the three BFS programs. Although UIUC-BFS is still much faster than IIIT-BFS, its running time is close to or even longer than the running time of CPU-BFS. The result shows that the highly imbalanced problems cannot benefit as much from the parallelism as the relatively balanced ones

Note that our GPU implementation pre-allocates the shared memory space for W-Frontiers and B-Frontiers based on the maximum node degree. If the maximum degree is much larger than the average degree, the memory usage becomes very inefficient. Thus, when handling very irregular graphs, we first converted them into near-regular graphs by splitting the big-degree nodes. A similar idea was used in [48]. This is a legitimate solution because most of the applications run BFS on the same graph a great number of times (if only run once, the BFS process is not really slow), and we only need to do the conversion once.

4.5.2 Experiments on Fermi GPU

Recently NVIDIA has delivered a new generation of GPU. The new GPU has a different architecture (called Fermi) than the GT200 GPU we have been working on. The most significant change is that two warps in the same

Table 4.2: BFS results on real world graphs

	#Vertex	IIIT-BFS (ms)	CPU-BFS (ms)	UIUC-BFS (ms)	Sp.
New York	264,346	79.9	41.6	19.4	2.1
Florida	1,070,376	372.0	120.7	61.7	2.0
USA-East	3,598,623	1471.1	581.4	158.5	3.7
USA-West	6,262,104	2579.4	1323.0	236.6	5.6

Table 4.3: BFS results on scale-free graphs

#Vertex	IIIT-BFS (ms)	CPU-BFS (ms)	UIUC-BFS (ms)
1M	161.5	52.8	100.7
5M	1015.4	284.0	302.0
10M	2252.8	506.9	483.6

Table 4.4: BFS results on Intel Xeon CPU/NVIDIA GeForce GTX480 GPU

#Verte	IIIT-BFS (ms)	CPU-BFS (ms)	UIUC-BFS (ms)	Speedup
1M	198.0	56.7	43.9	1.3
2M	489.4	124.5	68.1	1.8
5M	1725.8	356.0	131.3	2.7
7M	2786.3	508.8	171.8	3.0
9M	3981.3	658.1	213.1	3.0
10M	4626.5	737.0	231.2	3.1

block can be working simultaneously. However, our BFS solution can still be migrated to this new architecture. The correctness is guaranteed by the atomic operations on the W-Queues. And our hierarchical strategy can still greatly reduce the collision on the queues, although not completely avoid the collision. (There is collision on the W-Queues when threads from two warps are writing to the same W-Queue.)

We still perform our experiment on the NCSA AC cluster [46]. The new cluster node has one NVIDIA GeForce GTX480 graphics card with 1.5 GB global memory. Along with the new graphics card, the new cluster node also has a more advanced CPU chip, Intel Xeon 3.33 GHz processor with 12 M cache. We have tested the near-regular graphs of Table 4.1 on the new cluster node. The experimental results are shown in Table 4.4.

Compared with Table 4.1, both the CPU run times and the GPU run times have been greatly improved. However, more improvement is seen on the CPU than on the GPU. This is because the new CPU has much bigger cache than the old one, which plays an important role in the memory-bound applications such as BFS. This result further proves that it is extremely difficult to achieve great acceleration for the graph algorithms on the GPU. For one thing, the memory access for the graph structure is random and we cannot achieve global memory coalescing. Hence in our implementation, we choose to store

the graph structure in the texture memory. Still it is very hard for the small cache of texture memory to compete with several million bytes of cache on the CPU. In addition, almost all the computation in our BFS problem is integer computation, in which the GPU is not as superior to the CPU as in floating point arithmetics. However, we can still achieve moderate speedup on the GPU with well-designed data structure and algorithm.

4.6 Conclusion

We have presented a BFS implementation on the GPU. This work is most suitable for accelerating sparse and near-regular graphs, which are widely seen in the field of EDA. Both methods proposed in this chapter – hierarchical queue management and hierarchical kernel arrangement – are potentially applicable to the GPU implementations of other types of algorithms, too.

CHAPTER 5

GPU IMPLEMENTATION OF R-TREE

5.1 Introduction

The R-tree is an index data structure for the efficient management of spatial data such as rectangles. For example, to manage the shaded rectangles in Fig. 5.1, we can construct an R-tree as shown in Fig. 5.2. One of the most common operations on an R-tree is the range query: given a query rectangle, retrieve all the data rectangles that intersect it. A lot of EDA problems can be formulated as range query problems. For example, in design rule checking (DRC), overlap checking is a range query problem; spacing checking can also be solved by range query if we extend the boundary of the checked object by the minimum spacing amount. Another example is dummy via insertion. To obtain all the legitimate via locations, we look for the intersections between the routing segments on the two layers. In extreme ultraviolet lithography (EUVL), we check the overlaps between patterns/layouts and the defects. The key idea is to assure the patterns/layouts on the blank where the defects do not impact the final wafer image [49]. To efficiently solve these query problems, we use the R-tree structure.

There have been a lot of parallel works on R-tree query on different multiple-processor architectures [50], [51], [52], [53]. All of them tried to maximize the parallelism for large range queries on the particular architectures. Some of them also managed to improve the throughput by engaging as few disk I/Os as possible. In this work, we assume the spatial data can fit into memory, which is reasonable given that memory is getting larger and cheaper. Also we want to improve parallel R-tree query on the GPU. As far as we know, there is only one existing R-tree work on the GPU [54]. However, no speedup result is shown in [54]. Moreover, the work in [54] is not realistic by assuming the R-tree can be completely loaded into the GPU shared memory, which is

only 16 KB. Later on, we will also show that the implementation introduced in [54] does not fully explore the parallelism in R-tree query. Hence, our work is the first one which successfully uses the GPU to improve the speed of R-tree query.

There are typically two ways to construct an R-Tree. The first one is to repeatedly insert each data item. This method is often used when the database is frequently changing. Sometimes the database is static, such as in the above-mentioned applications in EDA. In this case, it is preferable to insert all of the data items using a single operation, or bulk loading [55]. In this work, we have also implemented a parallel bulk loading algorithm on the GPU. In the following discussion, we use R-tree construction and bulk loading interchangeably. Previous works on parallel R-tree construction [50], [56], [57] choose to partition the data set into several groups, build small R-trees in parallel and then combine the small R-trees into the final R-tree. They have faced the problems of choosing an appropriate partition algorithm, balancing the heights of individual R-trees, optimizing the data transfer among different processors and so on. In our implementation, we take a completely different way to parallelize the R-tree construction. We only build a single R-tree and parallelize the two basic operations in R-tree construction: sorting and packing. This method is a better fit for the massive parallel architecture of the GPU. Also without partitioning the data to individual R-trees, we can guarantee the same quality of R-tree construction as the sequential implementation.

In the following sections, we will first review R-trees and the bulk-loading process. Then we will introduce the GPU implementation of R-tree query and R-tree construction in Section 5.3 and Section 5.4 respectively. We will present experimental results in Section 5.5 and discuss the application of R-tree query to the via dropping problem in Section 5.6. The chapter is concluded with some final remarks in Section 5.7.

5.2 Background

R-tree is one of the most promising spatial data structures [58]. It is the extension of the B-tree for multidimensional objects. A geometric object is represented by its minimum boundingbox rectangle or MBR. Each leaf

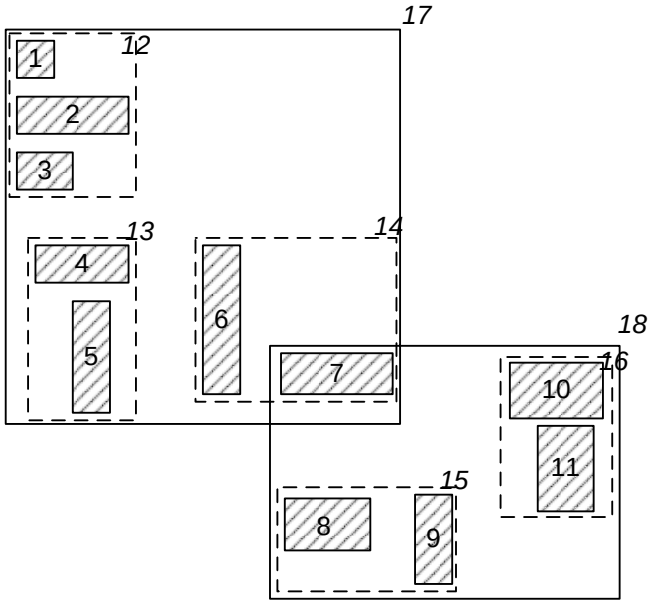


Figure 5.1: Data (shaded rectangles) organized in an R-tree with fanout 3.

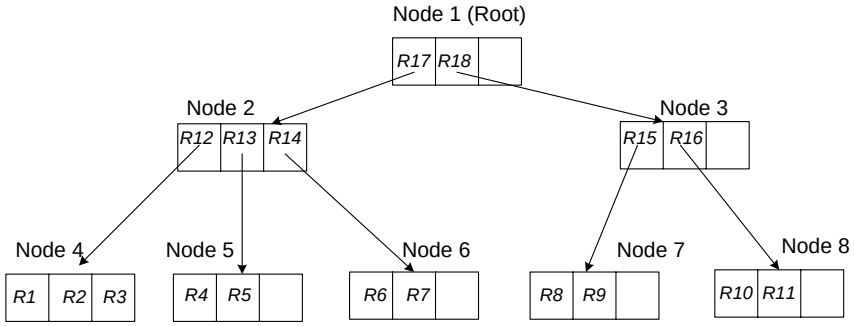


Figure 5.2: R-tree structure corresponding to Fig. 5.1.

node of an R-tree holds two items for each data record. One is the MBR of the record, and one is a pointer (or an identifier) to the data record itself. In the following discussion, we will assume each data object is simply a rectangle. Therefore, we use the terms *MBR of the data record* and *data rectangle* interchangeably. Similarly, each non-leaf node of an R-tree holds two items for each of its children: the MBR that covers all the rectangles in the child, and a pointer to the child. The main innovation in the R-tree is that the nodes (to be precise, the MBRs of nodes) are allowed to overlap. For example, in Fig. 5.2, nodes 2 and 3 are overlapping. This way, the R-tree can guarantee at least 50% space utilization and remain balanced. Figure 5.1 illustrates data rectangles (shaded rectangles) organized in an R-tree with fanout 3. The *fanout* of an R-tree is defined as the maximum number of children of each non-leaf node or the maximum number of data records in each leaf node. And Figure 5.2 shows the file structure for the same R-tree.

An R-tree could be constructed by repeated insertion of each data item. This approach does not require all the data items be available when the R-tree is built. However, in a lot of applications we know all the data beforehand. In this case, a better way is to do bulk loading, as it not only reduces the space overhead by packing the R-tree nodes as full as possible, but also offers better response time if a well-designed packing technique is employed. There are two types of approaches for bulk loading. The first one is called a bottom-up approach such as [59] and [60], which builds the R-tree level by level starting with the leaf level. The other one is a top-down approach [61], [55], which also builds the tree by levels, but starts with the root node. The authors of [55] have provided comparison between the two types of approaches. Nevertheless, since both types of approaches are composed of the same basic steps, sorting and packing, these approaches are not fundamentally different when it comes to parallel implementation. In our work, we implement a parallel version of bottom-up R-tree construction. The parallel ideas can be migrated to the top-down approaches.

Roussopoulos and Leifker [59] proposed the first method of bottom-up bulk loading. The idea is to sort the data on the x or y coordinate of one of the corners of the rectangles. The sorted list of rectangles is scanned; successive rectangles are assigned to the same R-tree leaf node, until this node is full; then, a new leaf node is created and the scanning of the sorted list continues. Thus, the nodes of the resulting R-tree will be fully packed,

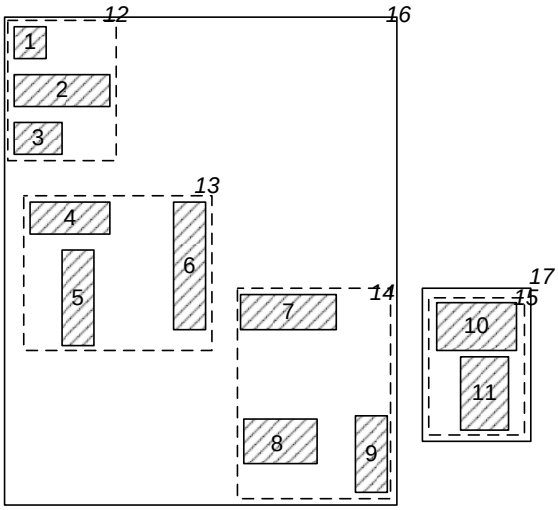


Figure 5.3: Data (shaded rectangles) organized in a lowx packed R-tree.

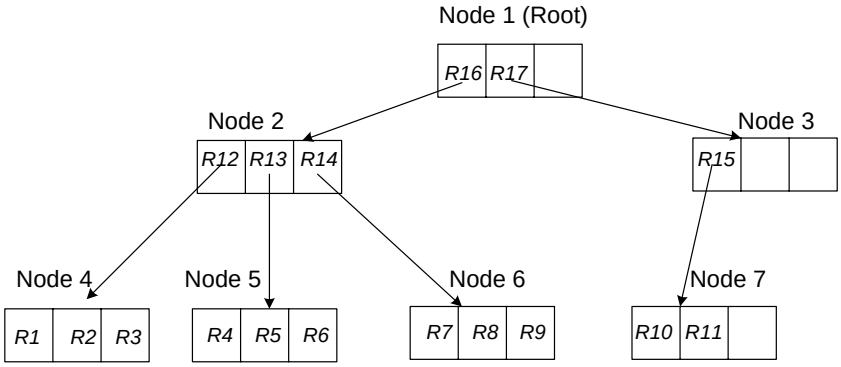
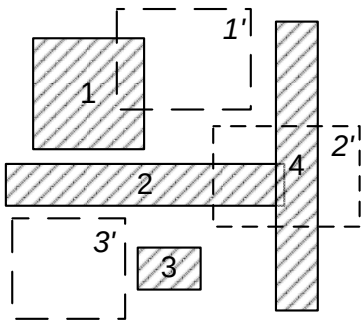


Figure 5.4: R-tree structure corresponding to Fig. 5.3.

with the possible exception of the last node at each level. The same process is applied to the MBRs of the leaf nodes to build another level of the R-tree. This process is applied iteratively until the root node of the R-tree is obtained. Fig. 5.3 shows how the data rectangles are packed using this approach. Here the rectangles are sorted on the x coordinate of the bottom left corner. This approach is referred to as *lowx pack* in [60]. And Fig. 5.4 is the corresponding R-tree structure of Fig. 5.3. Kamel proposed a better way to sort the data based on the Hilbert curve in [60]. However, since our parallelization method is not dependent on the sorting keys, we use the lowx pack method to study the R-tree construction technique in this work.



Input:
Data-Rects: {1, 2, 3, 4}
Query-Rects: {1', 2', 3'}

Output:
Inter-Result: {(1,{1}), (2,{2,4}), (0,{})}

Figure 5.5: An example of R-tree query problem.

5.3 Parallel R-tree Query

In this section, we discuss the GPU implementation of R-tree query. Note that our R-tree query method proposed here is independent of the R-tree construction methods.

5.3.1 R-tree Data Structures on the GPU

Given a list of data rectangles *Data-Rects* and a list of query rectangles *Query-Rects*, the R-tree query problem is to find a list of intersection results *Inter-Result*. The ith item in the resulting list has the form of:

$$(\#Intersection, list_of_data_rectangles)$$

Here the first element $\#Intersection$ is the number of data rectangles that intersect with the ith query rectangle. The second element is the list of identifiers of the intersecting data rectangles. Fig. 5.5 shows an example R-tree problem. Here, all the data rectangles are represented by shaded rectangles with solid boundaries and all the query rectangles are represented by clear rectangles with dashed boundaries.

We need two arrays to store the R-tree structure in the GPU memory. The first is an array of node structures called *Index*. The second is an array of

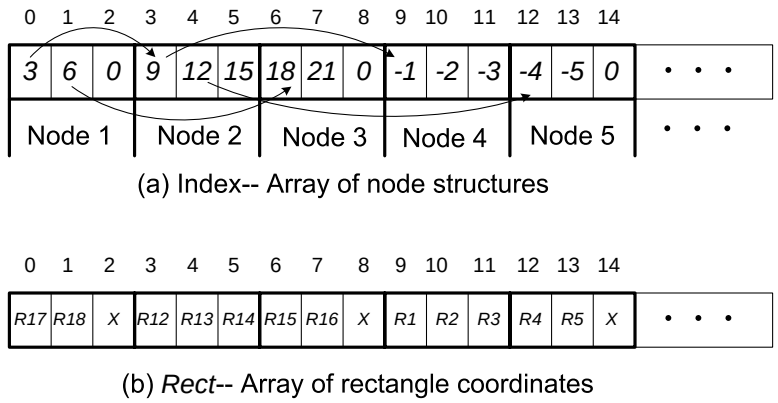


Figure 5.6: The GPU data structure of the R-tree in Fig. 5.2.

rectangle coordinates, called *Coord*. The size of each array is equal to the number of R-tree nodes multiplied by the fanout of the R-tree. Take Fig. 5.2 for example; we have 8 R-tree nodes and the fanout is 3, so the sizes of the two arrays are both equal to 24; each R-tree node is corresponding to 3 consecutive array items. Fig. 5.6 shows part of the arrays corresponding to the R-tree in Fig. 5.2. The numbers on the top of Fig. 5.6(a) and (b) are the array indices.

For an arbitrary array item $Index[(i-1) \times 3 + (j-1)]$ with $j = 1, 2, 3$ and $i = 1, 2, ...$, if Node i is a non-leaf node, then this array item represents the starting index of the jth child in the *Index* array; if Node i is a leaf-node, then this array item is the opposite number of the identifier of the node's jth data rectangle. For example in Fig. 5.6, $Index[0]$ is equal to the starting index of the first child of Node 1. $Index[12]$ is equal to -4, because the 1st rectangle of Node 5 is the 4th input data rectangle. We use 0 to represent both an empty child and a non-existing data rectangle. Each array item $Rect[(i-1) \times 3 + (j-1)]$ where $j = 1, 2, 3$ and $i = 1, 2, ...$ is a vector, representing the coordinates of the left-bottom and the right-top corners of the jth rectangle in Node i. We use a cross in Fig. 5.6(b) to represent *donot care*. We know the rectangle item does not exist because the corresponding item in the *Index* array is equal to 0.

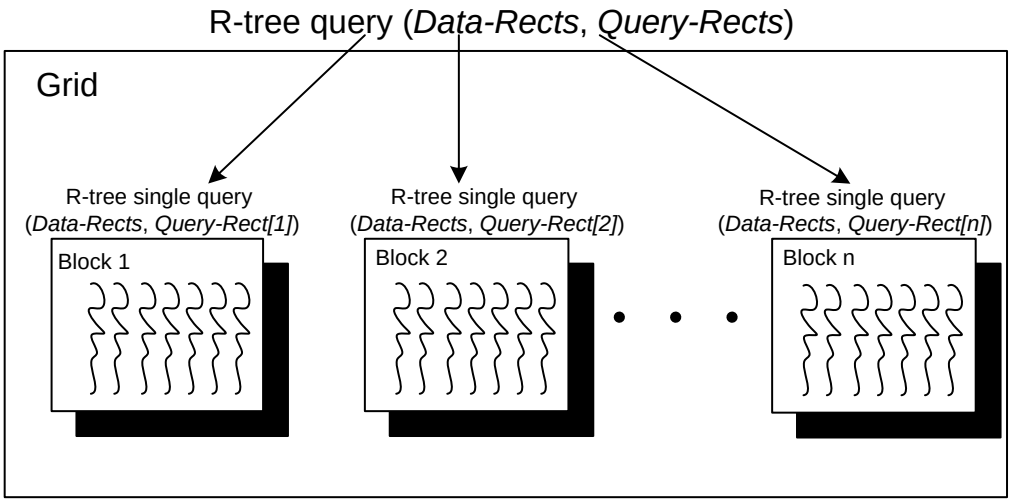


Figure 5.7: The two-level parallelism idea for R-tree query.

5.3.2 Parallelism Ideas

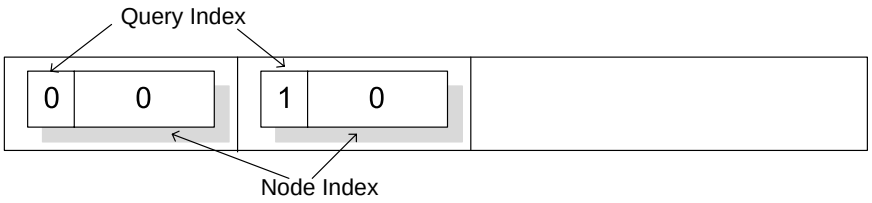
The R-tree query problem shows a great amount of parallelism and is a natural fit for the GPU architecture. The first level of parallelism is obvious. Assume the size of *Query-Rects* is n; we can easily achieve n-way parallelism by assigning one thread for each query rectangle. However, a single query on an R-tree (we will refer to this problem as *R-tree single query* problem) is too complicated for the light-weighted GPU thread. Hence, we need to explore another level of parallelism, using multiple threads to perform each query operation. Our overall idea is to follow the two-level thread hierarchy of the GPU. We launch n blocks to handle the R-tree query problem with n query rectangles. Then each block of threads is dedicated to solving one R-tree single query problem in parallel. Fig. 5.7 is the pictorial explanation of our two-level parallelism idea where each curve represents a thread.

The R-tree single query problem is very similar to a breadth-first search problem. We start with the root node and push it into a frontier queue. If the MBR of the root node does not intersect the query rectangle, we can terminate the search. Otherwise, we pop out the root node and push all the child nodes of the root node into the queue and go to the next level. In the next level, we pop out each frontier node and check its MBR against the query rectangle. Similarly if a frontier node intersects the query rectangle, we push the child nodes of the frontier node into the queue. This process continues until the queue becomes empty.

R-tree single query problem is different from the BFS problem in that we are given a tree structure as input instead of a graph structure. Also this tree structure is at least 50% full. Assume the fanout of the R-tree is d; 50% full means that each node has at least $\frac{1}{2}d$ non-empty children. We can take advantage of these properties and explore more parallelism. The GPU implementation of Chapter 4 is node-based parallelism, meaning that each thread is dedicated to one frontier node. Here to solve the R-tree single query problem, we employ child-based parallelism: each thread is dedicated to one child (including empty child) of a frontier node. Hence if the frontier has m nodes, we will have $d \times m$ threads and at least half of them are doing meaningful work.

Our implementation works better than the R-tree single query of [54] in two aspects. First, the implementation in [54] assumes the whole R-tree can fit into shared memory. We should know that the size of the shared memory is only 16 KB in current GPU device; therefore, this work is not realistic for large-scale problems. Second, node-based parallelism is used in [54], which explores less parallelism than our child-based method.

Note that the above implementation works best when each query rectangle has a great amount of overlap with the data rectangles. However, in a lot of EDA problems, such as the DRC problem, we usually see very few overlaps. In this case, the frontier within each block will typically be very small and we will not have enough threads to fully use all the computing resources or to hide the latency of memory access. To handle the sparse overlap problems, we propose a method to handle multiple query rectangles in one block by encoding the query index into the frontier queue. Fig. 5.8 demonstrates this idea. Here we handle two query rectangles in one block. We still only maintain one queue structure. But each queue element is composed of two parts. The highest bit indicates which query rectangle this queue element is corresponding to. The other bits indicate the node index in the R-tree. During the query process, every time we pop out an element from a queue, we use bit-wise operations to separate the query index and the node index. Then, we use the query index to locate the query rectangle we will be working on (the formula is shown in Fig. 5.8 (b)) and use the node index to locate the node in the R-tree. Thus we can achieve a fair amount of parallelism by working on several query problems simultaneously. The reason we want to encode the query index into the queue element instead of maintaining a



(a) Initial frontier queue

Query-Rect = Query-Rects[(block id) << 1+ (Query Index)]

(b) Formula to locate the query rectangle

Figure 5.8: Frontier queue structure handling two R-tree query requests in one block.

separate queue is that the queue operation on the GPU is very expensive (as analyzed in Chapter 4) but bit-wise computation is very cheap considering the high GFLOPS current GPUs can achieve.

5.3.3 Performance Discussion

In Section 5.3.1, we have presented the data structure to represent an R-tree in the GPU memory. Here we will discuss the performance consideration behind this specific representation. Obviously the *Index* and *Rect* arrays are obtained by traversing an R-tree in level-order, where we visit every node on a level before going to a lower level. This is called breadth-first traversal. This way we can guarantee that all the child nodes of one node are visited/stored continuously. This property is very important to achieve high memory access performance on the GPU. Since each child is corresponding to one thread, having all the child data together means that we can always guarantee memory coalescing. To guarantee fully coalescing, all the threads in the half warp need to access consecutive memory locations. Therefore, we prefer the fanout of the R-tree to be a multiple of the size of half warp on the GPU. On the contrary, if we store the R-tree by depth-first traversal, it will split the child nodes and cannot guarantee memory coalescing.

We can further improve the R-tree access performance by using the GPU constant memory. Note that in R-tree query, all the threads are accessing the same R-tree from top to bottom. Hence it will be helpful if we can take

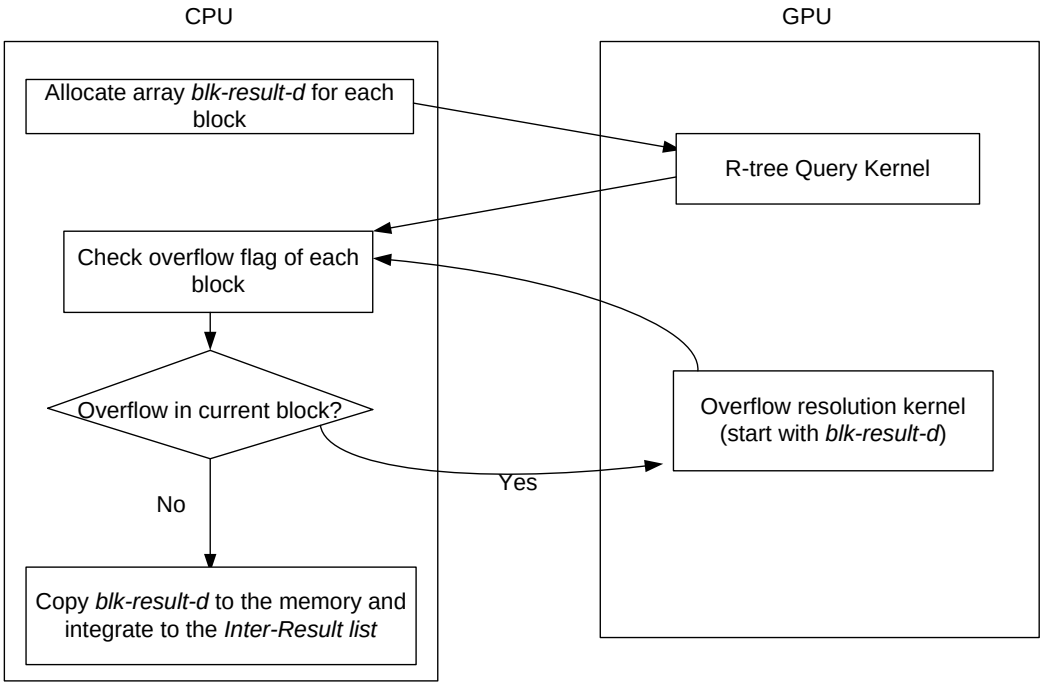


Figure 5.9: The flow of R-tree query. Here *blk-result-d* is a GPU global memory array, which stores the data rectangles that intersect the query rectangle(s) when no overflow happens or the frontier queue right before the overflow happens.

advantage of the cache of the constant memory. Of course, the constant memory is much smaller compared with the global memory and we can only store part of the R-tree in the constant memory. In implementation, we choose to store the top levels of the R-tree in the constant memory to maximize the cache hits. Experimental results show that by employing constant memory, we can reduce the run time by around 10%.

5.3.4 Handling Overflow Cases

The amount of overlap between each query rectangle and the data rectangles can vary a lot. In this section, we will discuss how to handle the imbalance issue on the GPU. For the convenience of explanation, we assume each block only handles a single query request.

Before we launch a kernel for R-tree query, we need to allocate the GPU global memory space to hold the R-tree query results and the shared memory space to hold the frontier queues. We can estimate the memory space

based on the property of the specific application or by sampling several query rectangles and evaluating the average amount of overlap. However, this estimation may not be accurate enough. We need to provide a mechanism to handle the overflow cases.

Based on the above analysis, the overflow happens on per-block basis. We know that in each block we do the R-tree query level by level. Therefore, if overflow happens, we can always go back to the previous level and store the status of the level right before the overflow. Then switch on the overflow flag for the current block. Once the R-tree query kernel is terminated, we can check the overflow flag of each block and handle the overflow case if there is any. We launch a new kernel to handle each overflow case. In this new kernel, we will start with the level right before the overflow and continue the remaining levels of query in parallel. Note that in the overflow resolution kernel, multiple blocks are dedicated to checking and storing the overlaps originally assigned to one block. Therefore, the chance of overflow has greatly reduced. But if we still have overflow, we can launch the overflow resolution kernel again, and eventually the overflow resolution can be guaranteed. Fig. 5.9 shows the overall flow of R-tree query.

5.4 Parallel R-tree Construction

We use the bulk loading algorithm proposed in [59] to construct an R-tree. Fig. 5.10 shows the pseudo code of this algorithm. Here we assume the fanout of the R-tree is d. This algorithm is called lowx pack, because the rectangles are sorted based on the x coordinates of the left bottom corners.

The bulk loading algorithm is composed of two building blocks. The first one is sorting (Step 4 in Fig. 5.10). The second one is the while loops (Step 5-8 and Step 10-13), or packing. In the following, we will discuss how to implement these two basic components on the GPU.

We use radix sort as the sorting engine on the GPU because it is a natural fit for parallelism and is among one of the most efficient sorting algorithms. Radix sort assumes that the keys are c-digit numbers and sorts on one digit of the keys at a time, from least to most significant. For a fixed key size, the complexity of sorting n input records will be $O(n)$. In our implementation, we employ the radix sort kernel provided by CUDA SDK [62]. Note that

```
1:  Let level l equal to 0
2:  repeat
3:    if l = 0 then
4:      Sort data rectangles on ascending x-coordinate of the left-
        bottom corner
5:      while more rectangles do
6:        generate a new R-tree node
7:        assign the next d rectangles to this node.
8:      end while
9:    else
10:     while more nodes in level l − 1 do
11:       generate a new R-tree node
12:       add the next d nodes of level l−1 as a child node of this new
          R-tree node
13:     end while
14:     if there is only one node in this level then
15:       mark this node as the root node
16:     end if
17:   end if
18: until get the root node
```

Figure 5.10: Pseudo-code of the bulk loading algorithm.

in the lowx pack algorithm we only need to sort the rectangles once at the leaf level. The MBRs in the higher levels are automatically sorted. If other sorting criteria are used, we may need to call the sorting procedure for every level.

Packing is another highly parallel operation because all the new R-tree nodes of the same level can be built in parallel. Most of the computation for packing comes from calculating the MBRs of the new R-tree nodes. To explore more parallelism, again we use child-based parallelism. Each child of the new R-tree node will be corresponding to one thread; then the MBR of the new R-tree node can be obtained by the reduction operations (max and min) on the MBRs of its child nodes. Fig. 5.11 illustrates the parallel packing process.

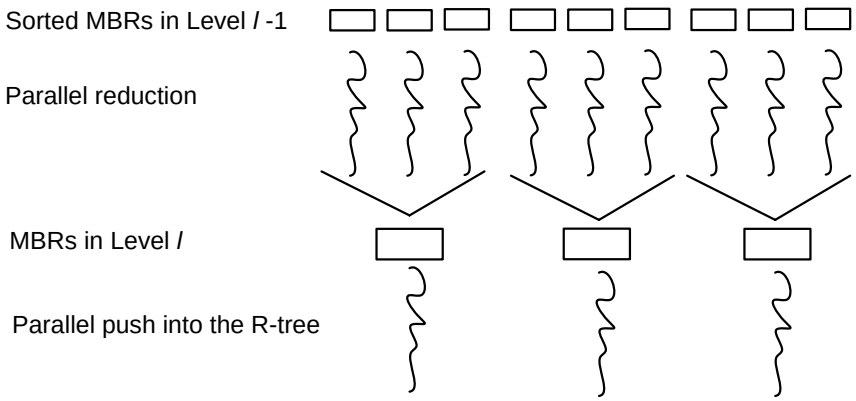


Figure 5.11: Parallel R-tree packing.

Table 5.1: Benchmark property

	#Data-Rects	#Query-Rects	Average Overlap	Maximum Overlap
Ex1	30,674	30,674	3	15
Ex2	76,999	76,999	3	17
Ex3	402,467	402,467	3	21
Ex4	2,249,727	2,249,727	5	358

5.5 Experimental Results

We conducted experiments on a host machine with 3.33 GHz Intel Xeon processor and 12 MB cache. A single NVIDIA GeForce GTX480 processor with 1.5 GB global memory was used to run CUDA applications. We have downloaded the test cases from [63]. Each test case is composed of a list of rectangles. We use them as both data rectangles and query rectangles. Table 5.1 shows the properties of these test cases. Column 1 is the number of data rectangles and Column 2 is the number of query rectangles, which are the same in our test. Column 3 is the average number of overlaps between each query rectangle and all the data rectangles. Column 4 is the maximum number of overlaps between each query rectangle and all the data rectangles. We have downloaded the R-tree sequential implementation from [64] as the baseline implementation.

Our first experiment is to see the speedup of R-tree query on the GPU. So we copy the R-tree generated by the sequential code [64] from the CPU to the GPU and only compare the R-tree query times. Table 5.2 shows the

Table 5.2: R-tree query time comparison

	CPU Query Time (ms)	GPU Query Time (ms)	Speedup
Ex1	62.90	2.79	X22.54
Ex2	180.03	6.36	X28.31
Ex3	1107.81	36.62	X30.25
Ex4	11594.00	331.52	X34.97

speedup results.

The GPU time reported in Table 5.2 includes the time to copy the query results back from the GPU memory to the CPU memory as well as the extra time to handle overflow cases. More speedup is achieved as the problem size grows. For the biggest benchmark, we can achieve almost 35 times speedup.

Our next experiment is to see how much speedup we can achieve in bulk loading. Note that the R-tree construction in [64] is done by repeatedly inserting nodes. So we modify the code in [64] and use the same lowx pack technique on the CPU. The CPU code of radix sort is obtained from [65]. Table 5.3 shows the run time comparison. Similar speedup results should be achieved if we use more complicated bulk loading algorithms such as the Hilbert curve based method [60], because the sorting process and the packing process are not fundamentally different in different bulk loading algorithms.

We did not show the query times on the R-trees constructed by the lowx pack algorithm, because the query time is misleading and is unfair for the CPU implementation. Generally the lowx pack algorithm tends to generate more overlaps in an R-tree than the well-designed construction method of [64]. These extra overlaps will affect the CPU query much more than the GPU query, because the GPU can compensate the time by using more parallelism. We believe the query speedups on R-trees constructed by better bulk loading algorithms should be close to the ones reported in Table 5.2.

5.6 Application to Via Dropping

The R-tree query technique can be used to accelerate the via dropping process in flip chip packaging. Flip chip is a method for interconnecting semiconductor devices, such as IC chips and micro-electro-mechanical systems (MEMs)

Table 5.3: R-tree construction time comparison

	CPU Construction Time (ms)	GPU Construction Time (ms)	Speedup
Ex1	12.56	3.71	X3.39
Ex2	31.90	5.65	X5.64
Ex3	179.23	13.62	X13.16
Ex3	1037.28	45.08	X23.01

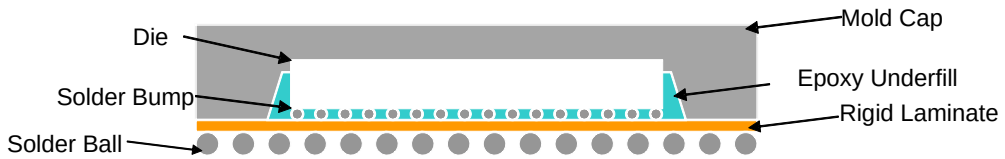


Figure 5.12: The flip chip package.

to external circuitry with solder bumps. The solder bumps can be deposited almost anywhere on top of the wafer during the final wafer processing step as long as the size and spacing constraints are satisfied. In order to mount the device to the circuit board, the device is flipped over so that the bumps can make the electrical and mechanical connections to the package. Compared with traditional carrier-based systems, the flip chip package directly sits on the circuit board and is much smaller than the carrier both in area and height. Also shorter wires are used to interconnect the chip to the external circuitry; therefore, flip chip allows higher-speed signals and carries heat better. Fig. 5.12 is the pictorial description of flip chip packaging. Here solder balls are corresponding to the pins in PCB routing problems introduced in the previous chapters.

In the packaging process, we introduce an extra routing layer above the top-metal layer, which is called redistribution layer or RDL (Fig. 5.13). RDL routing is to connect solder bumps to the pad openings of the die. A huge amount of routing on RDL as well as on the top metal is PG (power/ground) routing with the task of supplying power all over the die. In order to conduct more current, we want to drop as many vias as possible to connect the RDL and top metal routing. Apparently we can obtain all the legitimate via areas by computing the overlaps between the RDL wires and the top-metal wires of the PG net. According to the data we get from industry, there can

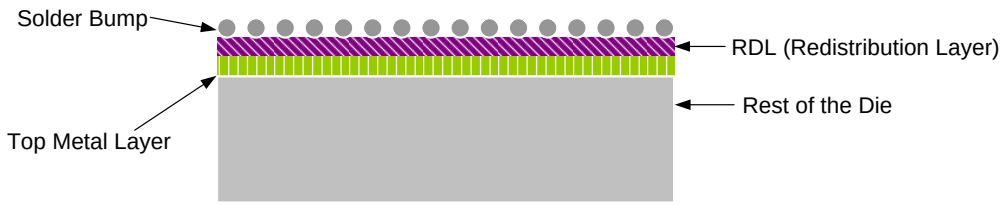


Figure 5.13: The redistribution layer and top-metal layer.

Table 5.4: properties of test chips

test chip	signal	#rdl-wires	#top-metal-wires
chip1	vdd	13,538	4,628
chip1	gnd	12,308	1,895
chip2	vdd	87,244	5,725
chip2	gnd	82,303	5,794

be thousands of PG wires on the top metal and tens of thousands of PG wires on RDL which can hold up to 100K vias or sometimes even millions of vias. To handle overlap calculation of this scale, R-tree is a preferred data structure.

We can use RDL wires as the data rectangles and top-metal wires as the query rectangles or the other way around. In our test cases, there are usually more RDL wires than top-metal wires, so the first option will take more time to build the R-tree but less time to finish the actual overlap query. Also if we need to adjust the routing in one layer frequently, it is better to use the wires in the other layer as the data rectangles. We extracted the RDL wires and top-metal wires from two real chip designs. The properties of these two test chips are summarized in Table 5.4. Table 5.5 shows the running times of computing overlaps between RDL and top-metal on both CPU and GPU (we use the same CPU/GPU here as in Section 5.5).

We can see that for the biggest test case, up to 16x speedup can be achieved. For the same net of the same test chip, more speedup can be achieved when using the top-metal wires as the data rectangles. This is mainly due to two reasons. First, more query rectangles (there are usually more RDL wires than top-metal wires) mean more parallelism on the GPU, hence more speedup. Second, using RDL wires as query rectangles creates a more uniform overlap situation (the gap between the maximum overlap and

Table 5.5: Time to calculate overlaps between RDL and top-metal routing

Case	#Data-Rects	#Query-Rects	Avg/Max Overlap	CPU (ms)	GPU (ms)	Speedup
chip1(vdd)	13,538	4,628	7/218	10.93	3.44	X3.18
chip1(vdd)	4,628	13,538	2/39	15.68	1.53	X10.24
chip1(gnd)	12,308	1,895	17/206	6.47	1.98	X5.83
chip1(gnd)	1,895	12,308	2/23	15.09	1.54	X9.80
chip2(vdd)	87,224	5,725	33/288	44.52	7.19	X6.06
chip2(vdd)	5,725	87,224	2/23	108.13	7.19	X15.04
chip2(gnd)	82,303	5,794	33/295	42.17	7.32	X5.76
chip2(gnd)	5,794	82,303	2/18	132.20	7.99	X16.55

Table 5.6: Time to construct R-trees for either the RDL wires or the top-metal wires.

Case	#Data-Rects	CPU (ms)	GPU(ms)	speedup
chip1(vdd)	13,538	6.05	3.76	X1.61
chip1(vdd)	4,628	2.12	2.96	X0.72
chip1(gnd)	12,308	5.48	3.47	X1.58
chip1(gnd)	1,895	0.88	3.41	X0.26
chip2(vdd)	87,224	39.44	6.24	X6.32
chip2(vdd)	5,725	2.53	3.04	X0.83
chip2(gnd)	82,303	36.23	5.96	X6.08
chip2(gnd)	5,794	2.52	3.04	X0.83

the average overlap is smaller), which favors the parallel architecture.

Table 5.6 shows the running time of R-tree construction on both the CPU and the GPU. Up to 6x speedup can be achieved when using bulk-loading to create R-trees. However, for small test cases, bulk loading on the GPU can be even slower than on the CPU. But the construction time for these cases is much less than the query time and is not the bottleneck of the via dropping problem.

Once we compute the physical overlapping area between RDL and top-metal routing, we can create vias in these areas. The actual via dropping needs to abide by various DRC rules; e.g., the vias should not overlap with the solder bumps, existing vias or other types of obstacles. We can also accelerate DRC by using the GPU based R-tree structure. Since the idea

of checking overlaps between vias and obstacles is very similar to checking overlaps between RDL and top-metal, we will not go into detail.

In this section, we have shown how to use GPU-based R-tree to accelerate the via dropping process. When we calculate the overlaps between two sets of rectangles, we always have the freedom to choose one set as data rectangles and the other as query rectangles. Usually we prefer to choose the relatively static set of rectangles as the data rectangles. Experiments on the data from industry show that we can always achieve good speedup on the R-tree query by using GPU. Speedup is also achieved for R-tree construction when the number of data rectangles becomes larger and the R-tree construction takes almost comparable time with the R-tree query. As the circuit design becomes larger and more complicated, we can expect more speedup by using the GPU.

5.7 Conclusion

In this chapter, we have presented the GPU implementations of R-tree query and R-tree construction. By using a two-level parallelism technique we can achieve up to thirty times speedup on R-tree query. Our method is general and robust. It can be applied to both dense and sparse overlap situations. Our implementation also provides a mechanism to handle overflow cases. Bulk loading is a preferred R-tree construction method when all the data are available beforehand. Bulk loading is a natural fit for GPU parallel implementation and more than twenty times speedup can be achieved over the sequential implementation.

CHAPTER 6

CONCLUSION

In this thesis, we have presented several new strategies for EDA problems. The first strategy (Chapter 2) is to apply a well-known approach – Boolean satisfiability to an untouched field – ordered escape routing. The second strategy (Chapter 3) is to design an unconventional methodology – boundary routing for the traditional routing problem. The third strategy is to use the new computing platform – GPU to accelerate the fundamental algorithm (Chapter 4) or data structure (Chapter 5) of EDA tools.

We have learned some valuable lessons about the balance of quality and speed during our research. Ideally we want every new strategy to improve both the design quality and the running time. Our boundary routing methodology has achieved this goal compared with the traditional shortest-path-based negotiated congestion router. However, we cannot always get the win-win situation. For example, the SAT router, although guarantees the optimal solution, can take too much time when the problem size grows larger. In this case, it is very important to find a good tradeoff between quality and speed by applying the divide-and-conquer technique or isolating the key problem. Of course, a more general solution will be exploiting more computing power brought by parallelization. Hence, our research on the GPU-based EDA algorithms.

The most attractive part about the GPU is that it provides the powerful computing resource at very low cost. But not every EDA algorithm is suitable for parallelization on the GPU. And the difficulty of migrating the CPU algorithm onto the GPU also varies from case to case. So we feel that the following two research directions are very promising in the future. One is to study better ways to program the EDA software on the GPU. Take the BFS algorithm for example; although it is very difficult to implement BFS on the GPU, we can still achieve fine speedup by carefully designing the data structure. The other direction is to exploit more affordable parallel

architectures which might be best suited to certain EDA problems. For example, the SAT solver can certainly be parallelized. Manolios and Zhang have proposed an incomplete SAT solver on the GPU [66]. But as pointed out in [67], the complete SAT solvers, which require significant control in the implementation of the non-chronological backtrack, are unsuitable for the computational paradigm of GPUs. Our B-Escape router also shows a decent amount of parallelism. The most trivial parallelism will be trying different boundary routing modes in parallel. This course-grained parallelism again is not suitable for the many-core architecture of the GPU, but may benefit from other multiprocessor architectures. Undoubtedly, parallel algorithms on different computing platforms have shown great potential in boosting the performance of EDA tools.

REFERENCES

[1] H. Harrer, D. M. Dreps, T.-M. Winkel, W. Scholz, B. G. Truong, A. Huber, T. Zhou, K. L. Christian, and G. F. Goth, "High-speed interconnect and packaging design of the IBM system z9 processor cage," *IBM Journal of Research and Development*, vol. 51, no. 1/2, pp. 37–52, 2007.

[2] T.-M. Winkel, W. D. Becker, H. Harrer, H. Pross, D. Kaller, B. Garben, B. J. Chamberlin, and S. A. Kuppinger, "First- and second-level packaging of the z990 processor cage," *IBM Journal of Research and Development*, vol. 48, no. 3-4, pp. 379–394, 2004.

[3] S. Green, "GPU physics," 2007. [Online]. Available: www.gpgpu.org/static/s2007/slides/15-GPGPU-physics.pdf

[4] K. Gulati, J. Croix, S. P. Khatri, and R. Shastry, "Fast circuit simulation on graphics processing units," in *Proc. IEEE/ACM Asia and South Pacific Design Automation Conference (ASP-DAC'09)*, Yokohama, Japan, Jan. 2009, pp. 403–408.

[5] [Online]. Available: http://www.gauda.com/

[6] I. Torunoglu, A. Karakas, E. Elsen, C. Andrus, B. Bremen, B. Dimitrov, and J. Ungar, "A GPU-based full-chip inverse lithography solution for random patterns," in *Proc. SPIE, Vol. 7641*, 2010, p. 764115.

[7] K. Gulati and S. P. Khatri, "Accelerating statistical static timing analysis using graphics processing units," in *Proc. IEEE/ACM Asia and South Pacific Design Automation Conference (ASP-DAC'09)*, Yokohama, Japan, Jan. 2009, pp. 260–265.

[8] K. Gulati and S. P. Khatri, "Towards acceleration of fault simulation using graphics processing units," in *Proc. IEEE/ACM Design Automation Conference (DAC'08)*, Anaheim, CA, June 2008, pp. 822–827.

[9] D. Chatterjee, A. DeOrio, and V. Bertacco, "GCS: High-performance gate-level simulation with GP-GPUs," in *Proc. Design Automation and Test in Europe (DATE'09)*, Nice, France, Apr. 2009, pp. 1332–1337.

[10] D. Chatterjee, A. DeOrio, and V. Bertacco, "Event-driven gate-level simulation with GP-GPUs," in *Proc. ACM/IEEE Design Automation Conference (DAC'09)*, San Francisco, CA, 2009, pp. 557–562.

[11] Z. Feng and P. Li, "Multigrid on GPU: Tackling power grid analysis on parallel SIMT platforms," in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD'08)*, San Jose, CA, Nov. 2008, pp. 647–654.

[12] Z. Feng and Z. Zeng, "Parallel multigrid preconditioning on graphics processing units (GPUs) for robust power grid analysis," in *Proc. ACM/IEEE Design Automation Conference (DAC'10)*, Anaheim, CA, June 2010, pp. 661–666.

[13] J. Shi, Y. Cai, W. Hou, L. Ma, S. X.-D. Tan, P.-H. Ho, and X. Wang, "GPU friendly fast Poisson solver for structured power grid network analysis," in *Proc. ACM/IEEE Design Automation Conference (DAC'09)*, San Francisco, CA, 2009, pp. 178–183.

[14] T. Chiba, M. Yamada, and F. Kobayashi, "Limitation of the signal pin density on wiring boards," *IEEE Transactions on Components, Packaging and Manufacturing Technology*, vol. 19, no. 2, pp. 391–396, May 1996.

[15] C. S. Patel, K. P. Martin, and J. D. Meindl, "Optimal printed wiring board design for high I/O density chip size packages," *Circuit World*, vol. 25, no. 4, pp. 25–27, 1999.

[16] J. Cong, J. Fang, and K.-Y. Khoo, "Via design rule consideration in multilayer maze routing algorithms," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 19, no. 2, pp. 215–223, Feb. 2000.

[17] A. Titus, B. Jaiswal, T. Dishongh, and A. Cartwright, "Innovative circuit board level routing designs for BGA packages," *IEEE Trans. Adv. Packaging.*, vol. 27, pp. 630–639, Nov. 2004.

[18] V. P. Roychowdhury, J. Bruck, and T. Kailath, "Efficient algorithms for reconfiguration in VLSI/WSI arrays," *IEEE Trans. Comp.*, vol. 39, no. 4, pp. 480–489, 1990.

[19] Y. Birk and J. B. Lotspiech, "On finding non-intersecting straightline connections of grid points to the boundary," *J. of Algorithms*, vol. 13, no. 4, pp. 636–656, Dec. 1992.

[20] W.-T. Chan and F. Y. Chin, "Efficient algorithms for finding maximum number of disjoint paths in grids," *Journal of Algorithms*, vol. 34, no. 2, pp. 337–369, 2000.

[21] J. W. Fang, I. J. Lin, P. H. Yuh, Y. W. Chang, and J. H. Wang, "A routing algorithm for flip-chip design," in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD'05)*, San Jose, CA, Nov. 2005, pp. 753–758.

[22] Y. Tomioka and A. Takahashi, "Monotonic parallel and orthogonal routing for single-layer ball grid array packages," in *Proc. IEEE/ACM Asia and South Pacific Design Automation Conference (ASP-DAC'06)*, Yokohama, Japan, Jan. 2006, pp. 642–647.

[23] J. W. Fang, C. H. Hsu, and Y. W. Chang, "An integer linear programming based routing algorithm for flip-chip design," in *Proc. ACM/IEEE Design Automation Conference (DAC'07)*, San Diego, CA, June 2007, pp. 606–611.

[24] T. Larrabee, "Efficient generation of test patterns using Boolean satisfiability," Ph.D. dissertation, Stanford University, Palo Alto, CA, Feb. 1999.

[25] T. Larrabee, "Test pattern generation using Boolean satisfiability," *IEEE Transactions on Computer-Aided Design*, vol. 11, no. 1, pp. 4–15, Jan. 1992.

[26] H. Konuk and T. Larrabee, "Explorations of sequential ATPG using Boolean satisfiability," in *Proc. IEEE VLSI Test Symposium*, Atlantic City, NJ, Apr. 1993, pp. 85–90.

[27] P. Stephan, R. K. Brayton, and A. Sangiovanni-Vincentelli, "Combinational test generation using satisfiability," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 15, no. 9, pp. 1167–1176, Sep. 1996.

[28] M. Velev and R. Bryant, "Effective use of Boolean satisfiability procedures in the formal verification of superscalar and VLIW microprocessors," in *Proc. ACM/IEEE Design Automation Conference (DAC'10)*, Las Vegas, NV, July 2001, pp. 226 – 231.

[29] S. Devadas, "Optimal layout via Boolean satisfiability," in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD'89)*, Santa Clara, CA, Nov. 1989, pp. 294–297.

[30] R. G. Wood and R. A. Rutenbar, "FPGA routing and routability estimation via Boolean satisfiability," *IEEE Trans. VLSI Systems*, vol. 6, no. 2, pp. 222–231, June 1998.

[31] G.-J. Nam, "An exploration of physical design problems via Boolean satisfiability (SAT)," presented at Ph.D. Forum at Design Automation Conference, New Orleans, LA, 1999.

[32] E. Goldberg, "Practical SAT solving: Achievements, problems and opportunities," presented at Algorithms for the SAT-problem Workshop, Berlin, Germany, 2006.

[33] N. Eén and N. Sorensson, "The minisat page," 2007. [Online]. Available: http://minisat.se/MiniSat.html

[34] M. M. Ozdal and M. D. Wong, "Simultaneous escape routing and layer assignment for dense PCBs," in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD'04)*, San Jose, CA, Nov. 2004, pp. 822–829.

[35] M. M. Ozdal, M. D. Wong, and P. Honsinger, "Simultaneous escape-routing algorithms for via minimization of high-speed boards," *IEEE Trans. on CAD of Integrated Circuits and Systems*, vol. 27, no. 1, pp. 84–95, 2008.

[36] L. McMurchie and C. Ebeling, "Pathfinder: A negotiation-based performance-driven router for FPGAs," in *Proc. ACM International Symposium on Field Programmable Gate Arrays (FPGA'95)*, Monterey, CA, Feb. 1995, pp. 111–117.

[37] C. Rodrigues, D. Hardy, J. Stone, K. Schulten, and W. Hwu, "GPU acceleration of cutoff pair potentials for molecular modeling applications," in *Proc. ACM International Conference on Computing Frontiers*, Ischia, Italy, May 2008, pp. 273–282.

[38] P. Harish and P. J. Narayanan, "Accelerating large graph algorithms on the GPU using CUDA," in *High Performance Computing (HiPC'07)*, Goa, India, Dec. 2007, pp. 197–208.

[39] Y. Deng, B. Wang, and S. Mu, "Taming irregular EDA applications on GPUs," in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD'09)*, San Jose, CA, Nov. 2009, pp. 539–546.

[40] T. Cormen, C. Leiserson, R. Rivest, and C. Stein, *Introduction to Algorithms*. Cambridge, MA: MIT Press, 2001.

[41] C. Lauterbach, M. Garland, S. Sengupta, D. Luebke, and D. Manocha, "Fast BVH construction on GPUs," *Computer Graphics Forum*, vol. 28, no. 2, pp. 375–384, May 2009.

[42] A. Yoo, E. Chow, K. Henderson, W. Mcledon, B. Hendrickson, and U. Catalyurek, "A scalable distributed parallel breadth-first search algorithm on Bluegene/L," in *Proc. ACM International Conference on Supercomputing*, Cambridge, MA, June 2005, pp. 25–32.

[43] D. Bader and K. Madduri, "Designing multithreaded algorithms for breadth-first search and ST-connectivity on the Cray MTA-2," in *Proc. International Conference on Parallel Processing (ICPP'06)*, Columbus, OH, June 2006, pp. 523–530.

[44] D. Scarpazza, O. Villa, and F. Petrini, "Efficient breadth-first search on the Cell/BE processor," *IEEE Trans. on Parallel Distributed Systems*, vol. 19, no. 10, pp. 1381–1395, Oct. 2008.

[45] S. Xiao and W. Feng, "Inter-block GPU communication via fast barrier synchronization," Dept. of Computer Science, Virginia Tech., VA, Tech. Rep. TR-09-19, Sep. 2009.

[46] 2009. [Online]. Available: http://iacat.uiuc.edu/resources/cluster

[47] C. Demetrescu, "9th DIMACS implementation challenge - Shortest paths," 2006. [Online]. Available: http://www.dis.uniroma1.it/∼challenge9/

[48] U. Meyer, "External memory BFS on undirected graphs with bounded degree," in *Proc. ACM-SIAM Symposium on Discrete Algorithms*, Washington, DC, Jan. 2001, pp. 87–88.

[49] J. Burns and M. Abbas, "EUV mask defect mitigation through pattern placement," in *Proc. SPIE, Vol. 7823, Poster Session*, Monterey, CA, Sep. 2010, p. 782340.

[50] B. Schnitzer and S. T. Leutenegger, "Master-client R-trees: A new parallel R-tree architecture," in *Proc. International Conference on Scientific and Statistical Database Management*, Cleveland, OH, July 1999, pp. 68–77.

[51] B. Wang, H. Horinokuchi, K. Kaneko, and A. Makinouchi, "Parallel R-tree search algorithm on DSVM," in *Proc. International Conference on Database Systems for Advanced Applications*, Hsinchu, Taiwan, 1999, pp. 237–244.

[52] H. S. Erik and G. Hoel, "Performance of data-parallel spatial operations," in *Proc. International Conference on Very Large Data Bases*, Santiago de Chile, Chile, Sep. 1994, pp. 156–167.

[53] I. Kamel and C. Faloutsos, "Parallel R-trees," in *Proc. ACM International Conference on Management of Data (SIGMOD'92)*, San Diego, CA, June 1992, pp. 195–204.

[54] M. Kunjir and A. Manthramurthy, "Using graphics processing in spatial indexing algorithms," 2009. [Online]. Available: http://dsl.serc.iisc.ernet.in/ mayuresh/graphics_report.pdf

[55] H. Alborzi and H. Samet, "Execution time analysis of a top-down R-tree construction algorithm," *Inf. Process. Lett.*, vol. 101, no. 1, pp. 6–12, Jan. 2007.

[56] A. Papadopoulos and Y. Manolopoulos, "Parallel bulk-loading of spatial data," *Parallel Computing*, vol. 29, no. 10, pp. 1419–1444, Oct. 2003.

[57] A. Cary, Z. Sun, V. Hristidis, and N. Rishe, "Experiences on processing spatial data with MapReduce," in *Proc. of the 21st International Conference on Scientific and Statistical Database Management*, New Orleans, LA, June 2009, pp. 302–319.

[58] A. Guttman, "R-trees: A dynamic index structure for spatial searching," in *Proc. ACM International Conference on Management of Data (SIGMOD'84)*, Boston, MA, June 1984, pp. 47–57.

[59] N. Roussopoulos and D. Leifker, "Direct spatial search on pictorial databases using packed R-trees," in *Proc. ACM International Conference on Management of Data (SIGMOD'85)*, Austin, TX, May 1985, pp. 17–31.

[60] I. Kamel, "On packing R-trees," in *Proc. International Conference on Information and Knowledge Management*, Washington, DC, Nov. 1993, pp. 490–499.

[61] Y. J. Garcia, M. A. Lopez, and S. T. Leutenegger, "A greedy algorithm for bulk loading R-trees," in *Proc. ACM International Symp. on Advances in Geographic Information Systems (GIS'98)*, Washington, DC, Nov. 1993, pp. 490–499.

[62] "NVIDIA CUDA SDK," 2009. [Online]. Available: http://www.nvidia.com/cuda

[63] Y. Theodoridis, "The R-tree-portal," 2003. [Online]. Available: http://www.rtreeportal.org

[64] [Online]. Available: http://superliminal.com/sources/sources.htm

[65] P. Terdiman, "Radix sort revisited," 2000. [Online]. Available: http://www.codecorner.com/RadixSortRevisited.htm

[66] P. Manolios and Y. Zhang, "Implementing survey propagation on graphics processing units," in *Proc. International Conference on Theory and Applications of Satisfiability Testing (SAT'06))*, Seattle, WA, Aug. 2006, pp. 311–324.

[67] J. Croix and S. Khatri, "Introduction to GPU programming for EDA," in *Proc. IEEE/ACM International Conference on Computer-Aided Design (ICCAD'09)*, San Jose, CA, Nov. 2009, pp. 276–280.